

Integrating storage location and order picking problems in warehouse planning

Allyson Silva^{a,*}, Leandro C. Coelho^{a,b}, Maryam Darvish^a, Jacques Renaud^a

^a Centre Interuniversitaire de Recherche sur les Réseaux d'Entreprise, la Logistique et le Transport (CIRRELT) and Faculté des Sciences de l'Administration, Université Laval, Québec, Canada

^b Canada Research Chair in Integrated Logistics, Université Laval, Québec, Canada

ARTICLE INFO

Keywords:

Storage location
Order picking
Warehouse
General Variable Neighborhood Search

ABSTRACT

Storage location and order picking are two interdependent problems arising in warehouse planning traditionally solved independently. We introduce and model the integrated storage location and order picking problem and four special cases with imposed routing policies (return, S-shape, midpoint and largest gap). Experiments show that these models are difficult to solve, even for small warehouses and few orders. Therefore, we present a General Variable Neighborhood Search metaheuristic, which is observed to be very efficient for those small instances. For larger warehouses and more pickings, we show that our metaheuristic significantly improves solutions generated by common storage policies.

1. Introduction

The most resource-intensive process performed in a warehouse is the order picking activity (Scholz and Wäscher, 2017), which is highly dependent on the storage location policy used. The storage location assignment may be performed immediately after the reception of a product or periodically. Its purpose is to place products in convenient locations where they can be easily picked during the picking process. Order picking refers to the activities performed to retrieve products from their storage locations to satisfy demands specified by customer orders. Due to labor intensity in such systems, the order picking process alone concentrates 50–75% of the total operating costs for a typical warehouse (Frazelle, 2016). Thus, warehouses are often optimized for cost-efficient order picking.

The problem of deciding where to locate products in a warehouse is known as storage location assignment problem (SLAP). It consists of determining the most efficient assignment of products to locations in order to minimize the total handling effort. The order in which the products are collected is determined by a routing strategy to be followed by the pickers in a problem known as order picking problem (OPP). The SLAP solution is an input of the OPP, since routes can be created only after product locations are known. At the same time, a SLAP solution can only be evaluated when the strategy to solve the OPP is known.

The SLAP and OPP as individual problems have been extensively studied (De Koster et al., 2007; Gu et al., 2007). When considered together, they are usually solved sequentially for different combinations of storage location and picking policies. These policies are usually very simple heuristics used to create a solution for each problem. They present advantages such as being easily memorized, reducing costs caused by pick errors, and they are easily accepted by the pickers (Petersen and Aase, 2004; Petersen et al., 2004).

* Corresponding author.

E-mail addresses: allyson.silva@cirrelt.ca (A. Silva), leandro.coelho@cirrelt.ca (L.C. Coelho), maryam.darvish@cirrelt.ca (M. Darvish), jacques.renaud@cirrelt.ca (J. Renaud).

<https://doi.org/10.1016/j.tre.2020.102003>

Received 30 October 2019; Received in revised form 20 April 2020; Accepted 31 May 2020

Available online 02 July 2020

1366-5545/ © 2020 Elsevier Ltd. All rights reserved.

Traditionally, SLAP is considered to be a tactical problem, while the OPP is an operational one. However, since the warehouse environment is dynamic, there is a need to continuously shift the location assignments of products. In a dynamic slotting strategy, the storage area of a warehouse is reorganized more frequently. Reassignments can occur, for example, on a daily (Kim and Smith, 2012) or monthly (Kofler et al., 2014) basis specifically for each pick wave or “period”. When multiple periods are considered, the storage area can be reorganized between each period in order to minimize the total travel distance by all pickings and reassignments. SLAP solutions are evaluated by calculating the exact routes that will be performed to retrieve the products demanded during a period. Thus, the storage allocations may be updated more frequently in order to optimize pickings performed during the considered period. Nevertheless, little attention had yet been given to integrating these two problems in warehouse planning (Van Gils et al., 2018c).

In this paper, we integrate the SLAP and OPP and solve them as a single problem that we name as *storage location and order picking problem* (SLOPP). We present a cubic mixed integer programming (MIP) model to solve the general integrated SLOPP, which does not assume any specific warehouse layout. However, we cannot ignore that heuristic routing policies are still widely used, so we also present MIP models to solve non-integrated versions of the SLOPP. In these versions, the picking is performed using well known heuristic routing policies as return, S-shape, midpoint and largest gap. We name these non-integrated versions, respectively, SL + Re, SL + Ss, SL + Mp and SL + Lg. As special cases of the integrated SLOPP, a feasible solution for any of them is clearly a feasible solution for the SLOPP itself. The four special cases are mathematically modeled considering the most common warehouse layout found in practice, i.e., the rectangular single-block layout with multiple aisles. We also propose a General Variable Neighborhood Search (GVNS) metaheuristic framework to approximate optimal solutions for the SLOPP and its special cases. Computational experiments attest to the effectiveness of the GVNS in solving the problems for the small instances. We also test it for regular size instances, as used in literature, to analyze the advantage of solving the integrated version of the problem. We show that optimizing storage location assignments can yield large gains in the picking performance when compared to arranging the storage locations using common storage policies, regardless of which routing policy is used to solve the OPP subproblem.

The remainder of this paper is organized as follows. In Section 2, we review the literature of the SLAP and the OPP. The MIP models for the general and integrated SLOPP and its four special cases are presented in Section 3. The proposed GVNS is presented in Section 4. Section 5 provides the computational experiments results. In Section 6 we discuss the practical implications of this research. Finally, our conclusions follow in Section 7.

2. Background on the storage location assignment and order picking problems

We present a brief review on the SLAP and OPP. For complementary information, we refer to the literature reviews in De Koster et al. (2007), Gu et al. (2007, 2010), Reyes et al. (2019) and Van Gils et al. (2018c).

2.1. Storage location assignment problem

Problems that involve the assignment of products to storage locations in the warehouse are known as SLAP. Several different performance measures to be optimized and constraints can be considered. The most commonly used is associated with the space and distance, which includes the minimization of travel distances (Reyes et al., 2019). This problem considers the storage area layout, the set of orders to be fulfilled and assumes a picking policy. The SLAP was introduced by Hausman et al. (1976) for an automated warehouse, and it has been widely studied since then (e.g., Carlo and Giraldo, 2012; Kofler et al., 2014).

The SLAP is related to the quadratic assignment problem (QAP), which is considered to be one of the hardest combinatorial optimization problems to solve optimally due to its nonlinear objective function (Drezner, 2008). In their review, Reyes et al. (2019) report that although exact methods exist to solve SLAP variants, such as those based in clustering and affinity-based assignment techniques (e.g., Chuang et al., 2012; Li et al., 2016; Ming-Huang Chiang et al., 2014), they usually are not used due to the complexity of the problem.

Since SLAPs are very hard problems to solve using exact methods, typically simple heuristics are used to generate the product assignments. These heuristics are typically divided into the random, dedicated, and class-based categories. This taxonomy was presented by Hausman et al. (1976) and it is still widely in use. The choice of the most appropriate policy depends on the available product information (Carlo and Giraldo, 2012; Gu et al., 2007).

The *random storage* consists of the random assignment of products to locations in the storage area. Its main advantage is its simplicity, but the downside is obviously the longer travel times for pickers to retrieve products (Chen et al., 2011; Kofler et al., 2014; Petersen and Aase, 2004). Tappia et al. (2019) also point that when fast moving products change rapidly over time, like in e-commerce warehouses, it is hard to implement other storage policies than the random one. The *dedicated storage* policy fixes the location of high-turnover products to “best” locations. Products are sorted using a demand based rule and locations sorted based on the distance to the I/O point and, then, the best products are assigned to the best locations. Several rules used to sort products and locations are discussed in literature (Kofler et al., 2014; Petersen, 1999; Van Gils et al., 2018b). The *class-based storage* policy is an alternative for when demand forecasting is possible but not very precise for each individual item. The class-based policy divides products into classes and storage locations into zones, then assigns each class to a zone. The final position of a product within its zone is usually determined randomly. A different way to classify items is by their affinity. Items that are frequently ordered together are said to be affine. Some studies (Mantel et al., 2007; Yener and Yazgan, 2019) use the QAP formulation to allocate products based on affinity, although not creating explicit classes. References for studies that use affinity-based rules in the classification of products are found in Kofler et al. (2014).

Fig. 1 provides an example of random and class-based policies (diagonal and within-aisle metrics). Squares with the same

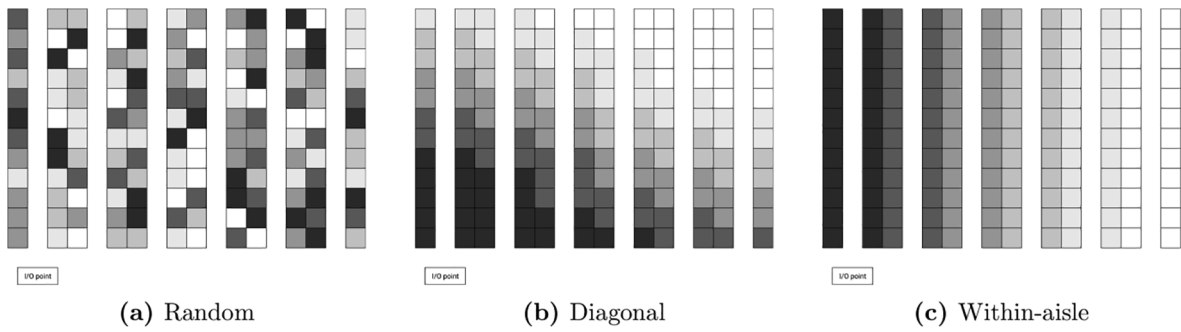


Fig. 1. Storage policies.

background color represent products with similar demand. We note that the random and the dedicated storage policies are special cases of the class-based one for when the number of classes is equal to one and to the number of products to be located, respectively.

2.2. Order picking problem

The objective of the OPP is to optimize a performance measure, e.g., minimize time or distance traveled by the picker. For pick lists with only one product, the OPP optimal solution is easily generated by solving a shortest path problem for the round trip between the I/O point and the product location. For a pick list with products in multiple locations, an optimal OPP solution is obtained by solving a special case of the Traveling Salesman Problem (TSP) (Applegate et al., 2007), given that there is enough capacity for the picker to retrieve all products in only one route. In the OPP, each item in the pick list, as well as the I/O point, corresponds to a vertex and the distance between two vertices is set to be the shortest distance between their corresponding locations in the warehouse. Formulations for the OPP adapted from the TSP literature are found in Pansart et al. (2018) and Scholz et al. (2016).

Any warehouse layout can be represented using a TSP formulation. Even when special conditions apply, like for narrow aisles (Chabot et al., 2018) or scattered warehouses (Weidinger, 2018). While the TSP is a hard problem to solve, the OPP can be solved efficiently under certain conditions. Ratliff and Rosenthal (1983) introduce an optimal algorithm for a rectangular single-block warehouse which is solvable in polynomial time using dynamic programming. The two-block case is described in Scholz and Wäscher (2017). Heuristics used for the TSP may be used to solve the OPP as well. The most successful one is the Lin-Kernighan-Helsgaun (LKH) (Helsgaun, 2000). Theys et al. (2010) used LKH to approximate optimal OPP routes reporting an average optimality gap of 0.1% for different warehouse settings.

While LKH searches for near-optimal OPP solutions, certain routing policies provide simple solutions that can be easily memorized and executed by the pickers. These routing policies are chosen according to the problem characteristics, such as warehouse shape, number of aisles and cross aisles, pick list size, and the storage and batching policies. Van Gils et al. (2018c) provided an extensive list of studies that analyze the interaction of these factors on the performance of the chosen routing policy.

Several routing policies are found in the OPP literature. The *return policy* considers that each picker enters and leaves through the same end of aisles containing pick items. In the *S-shape policy*, also known as traversal policy, a picker enters an aisle that contains a product to be picked and traverses it until its end. A return is allowed in the furthest location containing a pick of the last aisle visited, if it is advantageous (Roodbergen and De Koster, 2001). In the *midpoint policy*, pickers are able to go as far as the middle of the aisle before returning and leaving the aisle from where they entered. Exceptions are applied for the first and last aisles containing items so that the picker can access the opposite cross aisle and return to the I/O point after picking all items. In the *largest gap policy*, pickers enter aisles as far as to the point where the item that has the largest gap to another item, or the closest cross aisle, is located. Gaps are calculated as the distance between two adjacent items in the aisle or, if the item is the first or last item in the aisle, the distance between this item and the closest cross aisle. Then, the pickers return and leave at the same end that they entered. As in the midpoint policy, a picker never crosses the entire aisle except for the first and last ones containing items. Other policies include the combined and the aisle-by-aisle (Petersen, 1997; Roodbergen and De Koster, 2001). Examples of each of these routing policies are presented in Fig. 2, along with the optimal route and its cost in parentheses. Although most policies presented here are for a single-block layout, they are easily adaptable to multi-block cases (Roodbergen and De Koster, 2001).

2.3. Discussions on the interactions between SLAP and OPP

When analyzing the SLAP and OPP simultaneously, the main question raised is which OPP policy performs the best for a given SLAP policy, and vice versa. When deciding the route in the OPP, the product locations must be known prior to solving the problem. Nevertheless, the storage locations may be determined even if a routing strategy is not assumed *a priori*. Studies that analyze the interactions between the two problems consider that the SLAP and the OPP are solved in sequence. First, products are allocated to storage locations using a storage policy. Then, different routing policies are tested and their performances are evaluated to determine the overall performance of the system. Table 1 presents an overview of the studies that considered multiple policies for the SLAP and the OPP to analyze well-performing combinations under different circumstances.

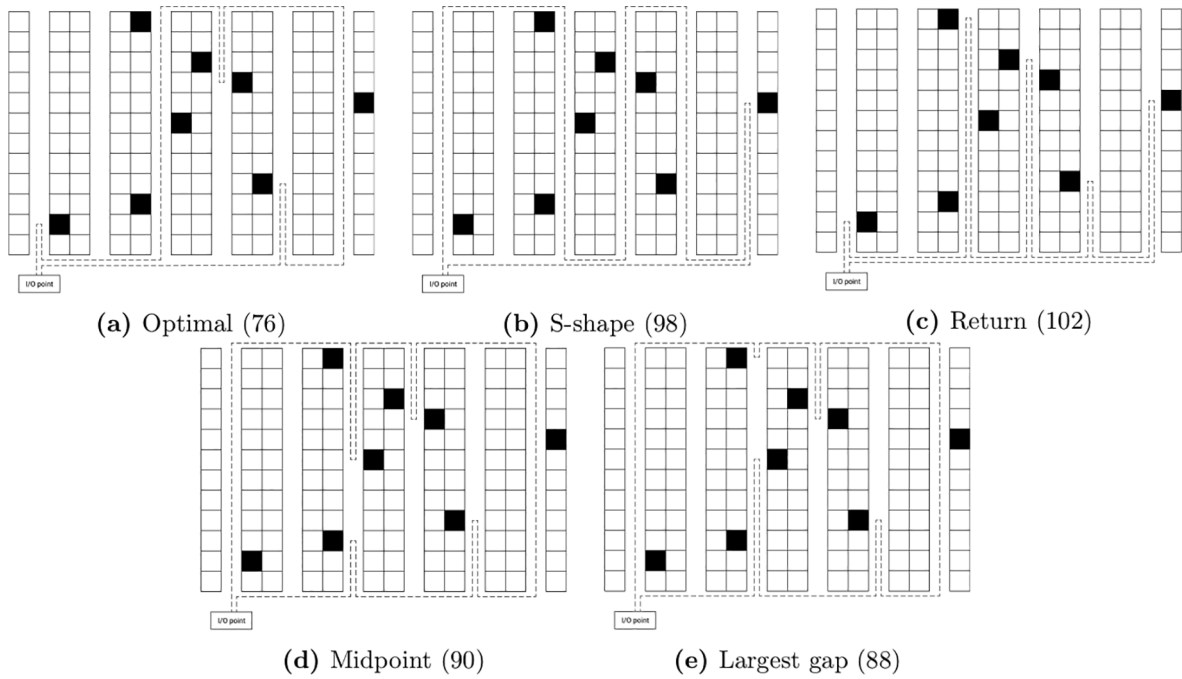


Fig. 2. Routing policies.

Table 1

Overview of the papers that investigated multiple storage and routing policies.

Reference	Storage policies					Routing policies	
	Random	Class-based		Dedicated		Optimal	Heuristic
		Demand	Distance	Demand	Distance		
Chan and Chan (2011)	✓	COI	Wa/Ot				Ss/Re/Ot
Chen et al. (2010)	✓	Fr	Wa/Ot				Ss/Re/Lg/Ot
Dekker et al. (2004)		Fr	Wa/Ot			✓	Ss/Mp/Lg/Ot
Dijkstra and Roodbergen (2017)		Fr	Di/Wa/Ot				Ss/Re/Mp/Lg
Hsieh and Huang (2011)	✓	Fr	Wa			✓	Ss/Ot
Hsieh and Tsai (2006)		Fr/Af	Di				Ss/Re
Pan and Wu (2012)	✓			Fr	Di/Wa/Aa		Ss/Re/Ot
Petersen (1999)	✓			Fr	Di/Wa	✓	Ss/Lg/Ot
Petersen and Aase (2004)	✓	Fr	Wa	Fr	Wa	✓	Ss/Ot
Petersen et al. (2004)				Fr	Di/Wa/Ot	✓	Ss
Quader and Castillo-Villar (2018)	✓	Fr	Wa/Ot				Ss/Re/Mp/Lg
Roodbergen et al. (2015)	✓	Fr	Di/Wa/Ot				Ss/Re/Lg/Ot
Theys et al. (2010)	✓			Fr	Wa	✓	Ss/Lg/Ot
Van Gils et al. (2018a)	✓	Fr	Wa/Ot				Ss/Re/Mp
Van Gils et al. (2018b)	✓	Fr	Di/Wa/Ot			✓	Ss/Re/Lg/Ot
Van Gils et al. (2019)	✓	Fr	Di/Wa/Ot			✓	Ss/Re/Mp/Ot

*Fr: Frequency-based, Af: Affinity-based, COI: Cube per order index, Wa: Within-aisle, Di: Diagonal, Ss: S-shape, Re: Return, Mp: Midpoint, Lg: Largest gap, Ot: Other.

When demands are known only at the product level, i.e., we have a good approximation of the number of times each product will be ordered, the best way to solve the SLAP is using a simple storage policy due to the uncertainties involved in the batching of orders to create pick lists. In this case, it is useful to know the interactions between storage and routing policies so that a good combination is used. Otherwise, when demands are known at the order level, i.e., we know not only the demand of each product but also the products contained in each pick list, it is possible to precisely evaluate SLAP solutions by calculating the exact routes that will be performed to retrieve the products in each picking route. This might be the case of markets with stable demands or those in which customers order with more antecedence, such that picking may be performed in waves (Kim and Smith, 2012). This approach is possible even on some unstable markets depending on the availability of data to forecast demands. In such situations, companies may decide to perform reassignment operations to better place the products, for example, in the beginning of the day or the week, in order to minimize picking effort during the period (Kofler et al., 2014).

All cited studies so far consider the decisions on both problems are made sequentially. The integration of location and routing decisions is not new and has proved to be beneficial both in warehousing (Van Gils et al., 2018b,c) and in general logistics problems (Nagy and Salhi, 2007; Salhi and Rand, 1989). If the routing policy is given as an input for the SLAP, and future orders are known or predictable, the storage area setup can be determined optimally for that policy. For instance, Boysen and Stephan (2013) model a SLAP with return routing using dynamic programming considering a layout with only one rack (half-aisle), also proving the NP-hardness of the case with a single aisle. Although some studies explore SLAPs with the assumption of a routing policy, only Mantel et al. (2007) and Dijkstra and Roodbergen (2017) consider a manual picker-to-parts warehouse with demands known at the product or order level, and optimal assignments at single storage locations level in a more complex layout than with a single aisle. They also compare solutions obtained for the optimal SLAP for different routing policies. However, Mantel et al. (2007) assume a specific warehouse system (vertical lift module) and present an MIP for the SLAP with an S-shape routing policy where a return is not allowed in the last aisle. Compared to them, we do not assume any specific warehouse system, and we present a mathematical model for the problem with a slightly different S-shape, besides other routing policies. Dijkstra and Roodbergen (2017) present a mathematical model to generate near-optimal storage location assignments using dynamic programming for the four routing policies presented here. The route lengths are calculated from picking probabilities for product retrieval instead of static pick lists as considered here. Their findings confirm the best combinations of storage and routing policies, e.g., return combines with diagonal, S-shape with within-aisle, and largest gap and midpoint both with within-aisle, among other storage policies. However, Dijkstra and Roodbergen (2017) did not evaluate the SLAP performance for the optimal OPP solution, which is one of the objectives of this study.

3. Mathematical models

In this section, we present MIP models to solve the general integrated SLOPP, as well as the four special cases considered here (SL + Re, SL + Ss, SL + Mp and SL + Lg). The general SLOPP model is derived from a TSP one. It is designed to accept any warehouse structure, so it is not required to know specific layout parameters *a priori*, such as the number of aisles, number of slots, etc. After, we adapt it to the special case of a rectangular single-block warehouse layout, which is also used as a reference to model the other four SLOPP special cases. Finally, the linearization technique used for all nonlinear models is briefly described.

3.1. General integrated SLOPP

A general formulation for the SLOPP is described next. The term *general* is used to indicate that the problem does not assume any storage layout. This is the most integrated and flexible version of the problem as it does not assume any policy or heuristic underneath. We consider a set $\mathcal{L} = \{1, \dots, L\}$ of locations representing the slots available, and $\mathcal{L}^* = \mathcal{L} \cup \{0\}$ the set including location 0 representing the I/O point. A set $\mathcal{P} = \{1, \dots, P\}$ of products to be located is defined, with $\mathcal{P}^* = \mathcal{P} \cup \{0\}$ being the set that includes a dummy product 0 that is assigned to the I/O point. Each product in $\mathcal{P}$ must be assigned to exactly one location in $\mathcal{L}$, thus $P \leq L$. Products not demanded by any order may be ignored from $\mathcal{P}$ without loss of generality of the model. A set $\mathcal{O} = \{1, \dots, O\}$ of orders is known, and each order o contains a pick list $Q_o \subset \mathcal{P}^*$, so that the products to be picked in each route are known in advance. Although we consider here a pick-by-order environment, where pick lists are composed solely of products from a single order, Q_o can also represent a previously defined batch of different customer orders that will be picked together without loss of generality. Since all routes start and end in the I/O point, the product 0 is contained in all pick lists. The matrix $\mathbf{D} = \{d_{ij}\}$ represents the cost (distance, time, etc.) for a picker to move from location i to location j . Note that values in $\mathbf{D}$ can be set to zero to represent products located in the same location. The binary routing variable x_{ij}^o is equal to one if the route to pick the items in the o -th order contains the path going from location i to location j , or zero otherwise. We assume that there is enough capacity for the picker to retrieve all products from o in only one route. The assignment variable y_{ki} is also binary and equals to one if product k is assigned to location i . $y_{00} = 1$ is defined to locate the I/O point at location 0. Finally, we define auxiliary variables u_k^o , which are used to create a valid route for order o , representing in which position product $k \in Q_o \setminus \{0\}$ is retrieved in the route. The general SLOPP is formulated as follows:

$$\text{Min} \sum_{o \in \mathcal{O}} \sum_{k \in Q_o} \sum_{l \in Q_o \setminus \{k\}} \sum_{i \in \mathcal{L}^*} \sum_{j \in \mathcal{L}^* \setminus \{i\}} d_{ij} x_{ij}^o y_{ki} y_{lj}, \quad (1)$$

subject to

$$\sum_{i \in \mathcal{L}} y_{ki} = 1, \quad \forall k \in \mathcal{P}, \quad (2)$$

$$\sum_{k \in \mathcal{P}} y_{ki} \leq 1, \quad \forall i \in \mathcal{L}, \quad (3)$$

$$\sum_{k \in Q_o} y_{ki} = \sum_{j \in \mathcal{L}^* \setminus \{i\}} x_{ij}^o, \quad \forall o \in \mathcal{O}, i \in \mathcal{L}^*, \quad (4)$$

$$\sum_{k \in Q_o} y_{kj} = \sum_{i \in \mathcal{L}^* \setminus \{j\}} x_{ij}^o, \quad \forall o \in \mathcal{O}, j \in \mathcal{L}^*, \quad (5)$$

$$u_k^o - u_l^o + |Q_o| x_{ij}^o y_{ki} y_{lj} \leq |Q_o| - 1, \quad \forall o \in \mathcal{O}, k, l \in Q_o \setminus \{0\}, k \neq l, i, j \in \mathcal{L}^*, i \neq j, \quad (6)$$

$$0 \leq u_k^o \leq |Q_o| - 1, \quad \forall k \in Q_o \setminus \{0\}, o \in O, \quad (7)$$

$$x_{ij}^o \in \{0, 1\}, \quad \forall o \in O, i, j \in \mathcal{L}^*, i \neq j, \quad (8)$$

$$y_{ki} \in \{0, 1\}, \quad \forall k \in \mathcal{P}, i \in \mathcal{L}^*. \quad (9)$$

The objective function (1) minimizes the total routing cost. Constraints (2) assign each product to exactly one location. Constraints (3) define that each location can have no more than one product assigned to it. Constraints (4) and (5) link the assignment and routing variables. They ensure that when there is a product k in order o located in a certain slot, then one path must be chosen that leaves and enters this slot. Constraints (6) eliminate subtours in the routes by guaranteeing that if product k is located in i , product l is located in j , and slot j is visited after slot i in the route, then $u_l^o > u_k^o$, where l and k are products in the pick list. These constraints are derived from the Miller-Tucker-Zemlin (MTZ) formulation for the TSP as presented in Scholz et al. (2016) for the OPP. Constraints (7) define the bound of the auxiliary variables, while constraints (8) and (9) define the domain of the routing and assignment variables, respectively.

We now describe the specific warehouse layout case examined in this paper. We consider a picker-to-product system for a rectangular warehouse with a single block. This is a common layout studied in literature (Dijkstra and Roodbergen, 2017; Petersen and Aase, 2004; Roodbergen and De Koster, 2001). The storage area has a set $\mathcal{A} = \{1, \dots, A\}$ of parallel aisles with a set $\mathcal{B} = \{1, \dots, B\}$ of rows of slots. Each row, including those in the first and last aisles, contains two slots, i.e., one in the left rack and another in the right rack. We ignore the distance between the two slots in the same row (two-sided picking). We denote (a, b) as the slot located in aisle a and row b . Due to the two-sided picking, we can ignore if it is on the left or right side of the row. A cross aisle exists both in the front and the back of the aisles. The distance a picker has to move from the entrance of an aisle to the entrance of an adjacent aisle is equal to M . Each slot has the capacity for a product with a sufficient number of single items meaning that no replenishment is needed during the picking. All products have the same physical requirements (space, temperature, etc.). Two neighbor rows within the same aisle have distance N , which is also the distance a picker has to travel to move from the head of the aisle in any cross aisle to the first row in the aisle. Pickers may perform a return anywhere at no cost, which is a common assumption when the picking equipment allows back and forth movement change with no additional effort. The effects of congestion is negligible, i.e., time to remove items is small enough to avoid pick-column blocking, and aisles allow picker passing each other, avoiding in-the-aisle blocking. Finally, an I/O point is located at the head of the first aisle. We illustrate this setting in Fig. 3. It represents the floor plan of a warehouse with $A = 5$ aisles and $B = 5$ rows in each aisle.

In order to solve the SLOPP for the single-block warehouse, the cost matrix $\mathbf{D}$ is determined by solving a special case of the shortest path problem between every pair of slots. The minimum distance d_{ij} between two slots located in $i = (a_i, b_i)$ and $j = (a_j, b_j)$ is calculated as:

$$d_{ij} = M(|a_i - a_j|) + \begin{cases} N|b_i - b_j|, & \text{if } a_i = a_j, \\ N \min\{(b_i + b_j), (2B + 2 - b_i - b_j)\}, & \text{otherwise.} \end{cases} \quad (10)$$

The first term gives the distance traveled in the cross aisles, while the second term gives the distance within-aisle either when both slots are in the same aisle or when they are in different aisles. The $\min$ function represents the decision of using the cross aisle in the front or in the back of the storage area to change aisles. The SLOPP may now be solved using the distance matrix as defined.

3.2. Storage location with different routing policies

It is also possible to model reductions of the SLOPP for the special case of a single-block warehouse when considering any routing policy other than the optimal one, which we refer to as *SL + routing policy*.

Consider D_o^{CA} as being the distance traveled by the picker in the cross aisles and D_o^{WA} as the distance traveled within the aisles to pick products from order o . For any routing strategy, the objective of the problem is to minimize the total distance traveled to fulfill

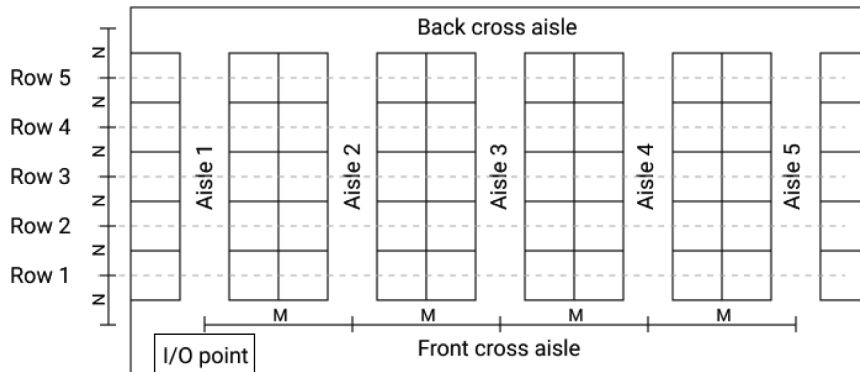


Fig. 3. Representation of a warehouse with two-sided parallel aisles in a single block.

Table 2
Notation of the models.

Variable	Description	Re	Ss	Mp	Lg
f_{oa}	Furthest row containing a pick in aisle a	✓	✓	✓	✓
z_{oa}	If there is a pick in aisle a	✓	✓	✓	✓
v_o	Last aisle containing a pick	✓		✓	✓
v_{oa}	If aisle a is the last aisle containing a pick		✓	✓	✓
k_o	Auxiliary variable used to calculate s_o^{SS}		✓		
s_o^{SS}	(S-shape special case) If the number of aisles with picks is odd		✓		
f_{oa}^1	Furthest row from lower cross aisle below midpoint containing a pick in aisle a			✓	
f_{oa}^2	Furthest row from upper cross aisle above midpoint containing a pick in aisle a			✓	
z_{oa}^1	If there is a pick in a row below midpoint in aisle a			✓	
z_{oa}^2	If there is a pick in a row above midpoint in aisle a			✓	
u_o^2	First aisle containing a pick above midpoint			✓	
u_{oa}^2	If aisle a is the first aisle containing a pick above midpoint			✓	
s_o^{MP}	(Midpoint special case) If $u_o^2 \neq 0$ and $u_o^2 \neq v_o$			✓	
w_{oa}^{MP}	If aisle a is neither the first aisle above midpoint nor the last aisle with a pick			✓	
G_{oa}	Largest gap between two neighbor picks or a pick and a neighbor cross aisle in aisle a				✓
g_{oab}	Gap between a pick in row $b \in B \cup \{0\}$ to nearest pick in another row or cross aisle in aisle a . When $b = 0$ we have the largest gap from the lower cross aisle to the first pick				✓
h_{oab}	Auxiliary variable used to calculate the largest gap				✓
u_o	First aisle containing a pick				✓
u_{oa}	If aisle a is the first one containing a pick				✓
s_o^{LG}	(Largest gap special case) If $u_o \neq v_o$				✓
w_{oa}^{LG}	If aisle a is between the first and last aisles with a pick				✓

all required orders given as

$$\min \sum_{o \in O} (D_o^{CA} + D_o^{WA}). \quad (11)$$

In order to calculate D_o^{CA} and D_o^{WA} , we should define the locations of the products to be picked in each route. Considering the set of products ($\mathcal{P}$) to be located, and the single-block layout as described in Section 3.1, we define the binary location variable y_{ab}^k as equal to one if product $k \in \mathcal{P}$ is assigned to the slot located in (a, b) , and zero otherwise. Thus, the following constraints can be used to assign all products in $\mathcal{P}$ to locations (constraints (12)) such that no location has more than two products (13), one on each side. Thus, D_o^{CA} and D_o^{WA} are functions dependent of y .

$$\sum_{a \in \mathcal{A}} \sum_{b \in \mathcal{B}} y_{ab}^k = 1, \quad \forall k \in \mathcal{P}, \quad (12)$$

$$\sum_{k \in \mathcal{P}} y_{ab}^k \leq 2, \quad \forall a \in \mathcal{A}, b \in \mathcal{B}, \quad (13)$$

$$y_{ab}^k \in \{0, 1\}, \quad \forall k \in \mathcal{P}, a \in \mathcal{A}, b \in \mathcal{B}. \quad (14)$$

For the sake of easiness to understand the models, we define new auxiliary variables to compute D_o^{CA} and D_o^{WA} for different routing policies instead of representing them by a function of y . We summarize in Table 2 the new auxiliary variables used in the four models we designed to solve the storage location (SL) with return (SL + Re), S-shape (SL + Ss), midpoint (SL + Mp), and largest gap (SL + Lg) routing policies, also indicating in which models they are used.

3.2.1. Storage location with return routing (SL + Re)

Considering the sets, variables, and parameters as previously defined, we represent the distances traveled in the SL + Re as follows. For any route $o \in O$, $D_o^{CA} = 2M(v_o - 1)$, i.e., the distance to travel back and forth between the I/O point and the last aisle containing a product to be picked. Furthermore, the distance traveled within all aisles is $D_o^{WA} = 2N \sum_{a \in \mathcal{A}} f_{oa}$, i.e., the sum of the distances to go back and forth from the front cross aisle to the furthest location that contains a pick in each aisle. In order to calculate v_o and f_{oa} , we added the following constraints to the base model (11)–(14) to create a model for the SL + Re:

$$f_{oa} \geq \sum_{b \in \mathcal{B}} b y_{ab}^k, \quad \forall o \in O, a \in \mathcal{A}, k \in Q_o \setminus \{0\}, \quad (15)$$

$$f_{oa} \leq B z_{oa}, \quad \forall o \in O, a \in \mathcal{A}, \quad (16)$$

$$\text{if } z_{oa} = 1 \text{ and } \sum_{c=(a+1)}^A z_{oc} = 0 \rightarrow v_o = a, \quad \forall o \in O, a \in \mathcal{A}, \quad (17)$$

$$z_{oa} \in \{0, 1\}, \quad \forall o \in O, a \in \mathcal{A}, \quad (18)$$

$$0 \leq f_{oa} \leq B, \quad \forall o \in O, a \in \mathcal{A}, \quad (19)$$

$$1 \leq v_o \leq A, \quad \forall o \in O. \quad (20)$$

Constraints (15) define a lower bound for f_{oa} . Since we minimize f_{oa} in the objective function, there is no need to limit its upper bound. Constraints (16) set z_{oa} to one if f_{oa} assumes a value greater than zero. z_{oa} is used to determine if a is the last aisle containing a product from order o , then $v_o = a$. These constraints can be written using indicator (or logical) constraints available in modern MIP solvers as presented in constraints (17). The remaining constraints (18)–(20) define the domain of the decision variables.

3.2.2. Storage location with S-shape routing (SL + Ss)

For the SL + Ss, the distances traveled are calculated as follows. The distance in the cross aisles is $D_o^{CA} = \sum_{a \in \mathcal{A}} 2Mv_{oa}(a - 1)$, which is similar to the return routing, but using the equivalent binary decision variables v . The use of a binary v is more adequate to calculate the distance within-aisles given as $D_o^{WA} = \sum_{a \in \mathcal{A}} (N(B + 1)z_{oa} + 2Ns_o^{SS}f_{oa}v_{oa}) - N(B + 1)s_o^{SS}$. The expression $N(B + 1)$ represents the distance traveled when the entire aisle is crossed by the picker. So, according to the first term of the equation, when there are products to be picked in an aisle, the picker crosses it entirely. Since this term also considers the last aisle independently of the value of s_o^{SS} , we have to add the last two terms to correct the distance traveled in the last aisle when the picker performs a return in it. Thus, the SL + Ss is modeled using the base model (11)–(14), constraints (15), (16), (18) and (19) to calculate z_{oa} and f_{oa} , and the following constraints to determine v_{oa} and s_o^{SS} :

$$\text{if } z_{oa} = 1 \text{ and } \sum_{c=(a+1)}^A z_{oc} = 0 \rightarrow v_{oa} = 1, \quad \forall o \in O, a \in \mathcal{A}, \quad (21)$$

$$\sum_{a \in \mathcal{A}} z_{oa} = 2k_o + s_o^{SS}, \quad \forall o \in O \quad (22)$$

$$v_{oa} \in \{0, 1\}, \quad \forall o \in O, a \in \mathcal{A}, \quad (23)$$

$$s_o^{SS} \in \{0, 1\}, \quad \forall o \in O, \quad (24)$$

$$0 \leq k_o \leq A/2, \quad \forall o \in O. \quad (25)$$

Constraints (21) are the equivalent indicator constraints to set the appropriate value for the binary variable v . The variable s_o^{SS} is calculated as the remainder of $\sum_{a \in \mathcal{A}} z_{oa}$ using constraints (22). Constraints (23)–(25) define the domain of the variables v_{oa} , s_o^{SS} and k_o , respectively. We note that our model is rather different than that of Mantel et al. (2007) because we consider the return option at the last aisle, which makes the problem significantly harder to solve.

3.2.3. Storage location with midpoint routing (SL + Mp)

To the formulation of the SL + Mp, we have divided the storage area into two zones. Midpoint is defined as the row $p = \lceil B/2 \rceil$. Zone 1 ($\mathcal{B}^1 = \{1, \dots, p\}$) contains all locations below p , including p itself. Meanwhile, zone 2 ($\mathcal{B}^2 = \{p + 1, \dots, B\}$) contains all locations above p . This definition is used to create variables z^1 and z^2 , which are the z equivalent of each zone, as well as f^1 and f^2 , which are their f equivalents. These, along with other exclusive auxiliary variables for this routing policy, are used to calculate the distances traveled as follows. As in the return routing, $D_o^{CA} = 2M(v_o - 1)$. Furthermore, $D_o^{WA} = 2N((B + 1)s_o^{MP} + \sum_{a \in \mathcal{A}} (f_{oa}^1 w_{oa}^{MP} + f_{oa}^2 w_{oa}^{MP} + f_{oa} v_{oa} - f_{oa} v_{oa} s_o^{MP}))$. This long equation can be explained by dividing it into three parts. The first part ($2N(B + 1)s_o^{MP}$) is the distance traveled to cross the first and last aisles entirely when s_o^{MP} is activated, i.e., there is at least one aisle with a product to be picked above the midpoint and it is different than the last aisle with a product to be picked. The second part ($\sum_{a \in \mathcal{A}} (f_{oa}^1 w_{oa}^{MP} + f_{oa}^2 w_{oa}^{MP})$) is the distance traveled to the rows closest to the midpoint in all aisles except for the first and the last. The third part ($\sum_{a \in \mathcal{A}} (f_{oa} v_{oa} - f_{oa} v_{oa} s_o^{MP})$) models the special case when there is only one aisle with products to be picked above midpoint and it is the last aisle. In this case, we ignore the zones and calculate the furthest row containing a product to be picked in the last aisle using variable f to determine the return point within the aisle. The SL + Mp is modeled using the base model (11)–(14), constraints (15) and (17) to calculate f_{oa} and v_o , and the following constraints to determine $f_{oa}^1, f_{oa}^2, z_{oa}^1, z_{oa}^2, z_{oa}, u_o^1, u_o^2, v_{oa}, s_o^{MP}$, and w_{oa}^{MP} :

$$\text{if } \sum_{b \in \mathcal{B}^r} \sum_{k \in Q_o} y_{ab}^k = 0 \rightarrow f_{oa}^r = 0, \quad o \in O, a \in \mathcal{A}, r = \{1, 2\}, \quad (26)$$

$$\text{if } \sum_{k \in Q_o} y_{ab}^k \geq 1 \text{ and } \sum_{d=(b+1)}^p \sum_{k \in Q_o} y_{ad}^k = 0 \rightarrow f_{oa}^1 = b, \quad o \in O, a \in \mathcal{A}, b \in \mathcal{B}^1 \quad (27)$$

$$\text{if } \sum_{k \in Q_o} y_{ab}^k \geq 1 \text{ and } \sum_{d=p+1}^{B-1} \sum_{k \in Q_o} y_{ad}^k = 0 \rightarrow f_{oa}^2 = (B + 1) - b, \quad o \in O, a \in \mathcal{A}, b \in \mathcal{B}^2 \quad (28)$$

$$\text{if } f_{oa}^r = 0 \rightarrow z_{oa}^r = 0, \quad o \in O, a \in \mathcal{A}, r = \{1, 2\}, \quad (29)$$

$$f_{oa}^r \leq Bz_{oa}^r, \quad o \in O, a \in \mathcal{A}, r = \{1, 2\}, \quad (30)$$

$$z_{oa} \geq z_{oa}^r, \quad o \in O, a \in \mathcal{A}, r = \{1, 2\}, \quad (31)$$

$$z_{oa} \leq \sum_{r=1}^2 z_{oa}^r, \quad o \in O, a \in \mathcal{A}, \quad (32)$$

$$\text{if } \sum_{a \in \mathcal{A}} z_{oa}^2 = 0 \rightarrow u_o^2 = 0, \quad o \in O, \quad (33)$$

$$\text{if } z_{oa}^2 = 1 \text{ and } \sum_{c=1}^{a-1} z_{oc}^2 = 0 \rightarrow u_o^2 = a, \quad o \in O, a \in \mathcal{A}, \quad (34)$$

$$\text{if } u_o^2 \neq 0 \text{ and } u_o^2 \neq v_o \rightarrow s_o^{MP} = 1, \quad o \in O, \quad (35)$$

$$\text{if } u_o^2 = 0 \text{ or } u_o^2 = v_o \rightarrow s_o^{MP} = 0, \quad o \in O, \quad (36)$$

$$u_o^2 = \sum_{a \in \mathcal{A}} au_{oa}^2, \quad o \in O, \quad (37)$$

$$\sum_{a \in \mathcal{A}} u_{oa}^2 \leq 1, \quad o \in O, \quad (38)$$

$$v_o = \sum_{a \in \mathcal{A}} av_{oa}, \quad o \in O, \quad (39)$$

$$\sum_{a \in \mathcal{A}} v_{oa} \leq 1, \quad o \in O, \quad (40)$$

$$w_{oa}^{MP} \leq 1 - u_{oa}^2, \quad o \in O, a \in \mathcal{A}, \quad (41)$$

$$w_{oa}^{MP} \leq 1 - v_{oa}, \quad o \in O, a \in \mathcal{A}, \quad (42)$$

$$w_{oa}^{MP} \geq 1 - u_{oa}^2 - v_{oa}, \quad o \in O, a \in \mathcal{A}, \quad (43)$$

$$0 \leq f_{oa}^1, f_{oa}^2 \leq p, \quad \forall o \in O, a \in \mathcal{A}, \quad (44)$$

$$z_{oa}^1, z_{oa}^2, z_{oa}, u_{oa}^2, v_{oa}, s_o^{MP}, w_{oa}^{MP} \in \{0, 1\}, \quad \forall o \in O, a \in \mathcal{A}, \quad (45)$$

$$0 \leq u_o^2 \leq A, \quad \forall o \in O. \quad (46)$$

Constraints (26)–(28) determine the furthest row to be traveled in each zone of each aisle. Constraints (29) and (30) set the proper values to variables z^1 and z^2 . Constraints (31) and (32) set the proper value to variables z . Constraints (33) and (34) determine the first aisle with products to be picked above the midpoint. Constraints (35) and (36) set s^{MP} to its appropriate value. Constraints (37) set proper values to the integer and binary versions of u^2 and constraints (38) certify that no more than one binary is activated. Constraints (39) and (40) do the same for v . Constraints (41)–(43) set w^{MP} to its correct value. Finally, constraints (44)–(46) define the domain of the variables.

3.2.4. Storage location with largest gap routing (SL + Lg)

The SL + Lg is modeled as follows. The distance traveled in the cross aisles is similar to the return and midpoint, given as $D_o^{CA} = 2M(v_o - 1)$. To calculate the distance traveled within aisles, like in the SL + Mp, we have to consider the different cases when only the lower cross aisle or both cross aisles are used. The first case is observed when all picked items are located in a single aisle. If more than two aisles should be entered, then the first and last are entirely crossed, while in the intermediary ones returns are performed to avoid the distance from the largest gap. Thus, the distance within-aisle is given as $D_o^{WA} = 2N((B+1)s_o^{LG} + \sum_{a \in \mathcal{A}} (s_o^{LG} w_{oa}^{LG}(B+1 - G_{oa}) + (1 - s_o^{LG})f_{oa} v_{oa}))$. The first term in the formula $((B+1)s_o^{LG})$ represents the first and last aisles crossed entirely when s^{LG} is active. The second term $(\sum_{a \in \mathcal{A}} (s_o^{LG} w_{oa}^{LG}(B+1 - G_{oa})))$ is the distance traveled in the intermediary aisles given by the total number of rows minus the largest gap in that aisle. The third term $(\sum_{a \in \mathcal{A}} ((1 - s_o^{LG})f_{oa} v_{oa}))$ is the special case when all picks are in the same aisle, so that a simple return at the furthest row containing a pick is performed. Now, we model the SL + Lg using the base model (11)–(14), constraints (15)–(17), (39), (40) to calculate f_{oa} , z_{oa} , v_o , v_{oa} , and the following constraints to determine u_o , u_{oa} , g_{oab} , G_{oa} , w_{oa}^{LG} , and h_{oab} :

$$\text{if } z_{oa} = 1 \text{ and } \sum_{c=1}^{a-1} z_{oc} = 0 \rightarrow u_o = a, \quad o \in O, a \in \mathcal{A}, \quad (47)$$

$$u_o = \sum_{a \in \mathcal{A}} au_{oa}, \quad o \in O, \quad (48)$$

$$\sum_{a \in \mathcal{A}} u_{oa} = 1, \quad o \in O, \quad (49)$$

$$\text{if } u_o \leq (a - 1) \text{ and } v_o \geq (a + 1) \rightarrow w_{oa}^{LG} = 1, \quad o \in O, a \in \mathcal{A}, \quad (50)$$

$$\text{if } u_o \geq a \text{ or } v_o \leq a \rightarrow w_{oa}^{LG} = 0, \quad o \in O, a \in \mathcal{A}, \quad (51)$$

$$\text{if } u_o = v_o \rightarrow s_o^{LG} = 0, \quad o \in O, \quad (52)$$

$$\text{if } u_o \neq v_o \rightarrow s_o^{LG} = 1, \quad o \in O, \quad (53)$$

$$\text{if } z_{oa} = 0 \rightarrow g_{oa0} = 0, \quad o \in O, a \in \mathcal{A}, \quad (54)$$

$$\text{if } \sum_{k \in Q_o} y_{oab}^k \geq 1 \text{ and } \sum_{d=1}^{b-1} \sum_{k \in Q_o} y_{oad}^k = 0 \rightarrow g_{oa0} = b, \quad o \in O, a \in \mathcal{A}, b \in \mathcal{B}, \quad (55)$$

$$\text{if } \sum_{k \in Q_o} y_{oab}^k = 0 \rightarrow g_{oab} = 0, \quad o \in O, a \in \mathcal{A}, b \in \mathcal{B}, \quad (56)$$

$$\text{if } \sum_{k \in Q_o} y_{oab_1}^k \geq 1 \text{ and } \sum_{k \in Q_o} y_{oab_2}^k \geq 1 \text{ and } \sum_{b_3=(b_1+1)}^{b_2-1} \sum_{k \in Q_o} y_{oab_3}^k = 0 \rightarrow g_{oab_1} = b_2 - b_1, \quad o \in O, a \in \mathcal{A}, b_1, b_2 \in \mathcal{B}, \quad (57)$$

$$\text{if } \sum_{k \in Q_o} y_{oab_1}^k \geq 1 \text{ and } \sum_{b_2=(b_1+1)}^B \sum_{k \in Q_o} y_{oab_2}^k = 0 \rightarrow g_{oab_1} = B + 1 - b_1, \quad o \in O, a \in \mathcal{A}, b_1 \in \mathcal{B}, \quad (58)$$

$$G_{oa} \geq g_{oab}, \quad o \in O, a \in \mathcal{A}, b \in \mathcal{B}, \quad (59)$$

$$G_{oa} \leq g_{oab} + \sum_{b_2 \in \mathcal{B} \cup \{0\}} (B + 1) h_{oab_2}, \quad o \in O, a \in \mathcal{A}, b \in \mathcal{B}, \quad (60)$$

$$\sum_{b \in \mathcal{B} \cup \{0\}} h_{oab} = 1, \quad o \in O, a \in \mathcal{A}, \quad (61)$$

$$u_o, g_{oab}, G_{oa} \geq 0, \quad \forall o \in O, a \in \mathcal{A}, \quad (62)$$

$$u_{oa}, w_{oa}^{LG}, h_{oab} \in \{0, 1\}, \quad \forall o \in O, a \in \mathcal{A}, b \in \mathcal{B}. \quad (63)$$

Constraints (47) determine the first aisle with a pick. Constraints (48) and (49) set proper values for u_o and u_{oa} . Constraints (50) and (51) use indicator constraints to ensure that $w_{oa}^{LG} = 1$ when $u_o < a < v_o$, and zero otherwise. Constraints (52) and (53) guarantee that $s_o^{LG} = 1$ only if the first and last aisles are different. If there is no pick in the aisle, then the gap is set to zero in constraints (54). Constraints (55) make sure that if there is a product in row b and there is no product between the lower cross aisle and row $b - 1$, then the gap from the lower cross aisle to the first pick is b . Constraints (56) set the gap in row b to zero when there is no pick in b . Constraints (57) state that if there is a pick in b_1 and b_2 , and no other pick between the two, then the gap in row b_1 is equal to $b_2 - b_1$. Constraints (58) handle the special case when the pick in b_1 is the last pick in the aisle. So, the gap in b_1 is the distance from it to the upper cross aisle. Constraints (59)–(61) are linear forms to compute the largest value between all gaps, i.e., $G_{oa} = \max(g_{oab} | b \in \mathcal{B} \cup \{0\})$. Finally, constraints (62) and (63) define the domain of the new variables.

3.3. Linearizations

The introduced models, except for the SL + Re, are nonlinear MIP formulations due to the product of variables either in the objective function or in constraints, as in (6). Nonlinear MIP models can be reformulated to an equivalent linear representation using a linearization technique. In order to solve the problems, we reformulated the models using the standard linearization technique (Glover and Woolsey, 1974). The technique is used to replace the product of variables as follows:

- product of two binary variables a and b is treated by replacing ab by a new variable c and adding the constraints $c \leq a$, $c \leq b$ and $c \geq a + b - 1$;
- product of three binary variables a , b , and c , abc is replaced by d and the constraints $d \leq a$, $d \leq b$, $d \leq c$ and $d \geq a + b + c - 2$;
- product of binary variable a and non-binary variable b , ab is replaced by c by adding the constraints $c \leq \bar{b}a$, where $\bar{b}$ is the upper bound value b can assume, $c \leq b$, and $c \geq b - \bar{b}(1 - a)$.

The advantage of linearizing a problem is that it can be solved by any integer linear programming solver. While the standard linearization benefits from its simplicity, it has the disadvantage of requiring a large number of auxiliary variables and constraints to replace the nonlinear terms.

4. A General Variable Neighborhood Search for the SLOPP and its special cases

Variable Neighborhood Search (VNS) is a metaheuristic that uses a systematic change of neighborhood within a descent phase to find a local optimum starting from different points generated by a perturbation phase (Mladenović and Hansen, 1997). Among its several variants, the one with some of the most successful applications is the General VNS (GVNS) (Hansen et al., 2019). In the GVNS, the descent phase consists in the application of several local searches (LS) performed in a deterministic way by a method called Variable Neighborhood Descent (VND). The reasoning of VND is that a local optimum in a neighborhood is not necessarily a local optimum in another, such that the use of several LSs are more likely to reach global optimum (Hansen et al., 2019). The implemented VND is shown in Algorithm 1.

We first define an initial solution, as described in Section 4.1. Then, a set N_l of neighborhoods is defined with $l = \{1, 2, 3\}$ representing the three LSs, presented in Section 4.2, used for the exploration, and their exploration order is known *a priori* (line 1). The first improving solution found in a neighborhood replaces the current one (*first improvement* strategy) until there is no possibility of improvement in that neighborhood (lines 4–6). Preliminary experiments showed that the first improvement strategy performed better than searching the whole neighborhood for the best improving solution which is too time consuming. The search returns to the first neighborhood when the solution is improved in the current neighborhood. Otherwise, it continues in the next neighborhood (lines 7–11). The solution returned by the VND (line 13) should be a local minimum with respect to all three neighborhoods.

Algorithm 1

Function VND(x).

```

1: Neighborhoods  $N_l(x)$ ,  $l = \{1, 2, 3\}$ ;
2: repeat
3:    $x_l \leftarrow x$ ;
4:   while there is a  $x' \in N_l(x) | f(x') < f(x)$  do
5:      $x \leftarrow x'$ ;
6:   end while
7:   if  $f(x) < f(x_l)$  then
8:      $l = 1$ ;
9:   else
10:     $l = l + 1$ ;
11:  end if
12: until  $l < 4$ 
13: return  $x$ .
```

The perturbation phase of the GVNS consists in applying a function *Shake* as presented in Algorithm 2. This function is used to move from solution x to a random solution in the *Slot Exchange* neighborhood. Preliminary experiments showed that the *Slot Exchange* neighborhood performs particularly well. In summary, each shake consists in selecting two random and different products and swap their locations in the storage area (more details about this neighborhood is presented in Section 4.2). A total of S swaps is performed before *Shake* ends. Note that small values of S tend to lead to the same local optimum when VND is applied. Otherwise, if S is too high, the new solution may lose the good characteristics of the initial solution, thus behaving like a randomly generated solution.

Algorithm 2

Function Shake (x , S).

```

1: for  $s = 1$  to  $S$  do
2:   Randomly select  $i, j \in \mathcal{P} | i \neq j$ ;
3:    $x \leftarrow \text{SlotExchange}(x, i, j)$ ;
4: end for
5: return  $x$ .
```

The steps of our GVNS are presented in Algorithm 3. Besides an initial solution x , it receives the maximum number of cycles K and shakes in each cycle S , and the time limit T . We start the GVNS by applying the VND to x (line 2). The stopping criteria are either when K or T is reached. In each cycle, we iterate a counter s until S is reached. For each s , a shake followed by a VND is performed (lines 5–6). If the objective function of the new local optimum improves the previous best known solution (BKS), s is reset, and the BKS is updated. Otherwise, s is incremented (lines 7–12). The shake counter is reset at the end of each cycle (line 15).

Algorithm 3Function GVNS (x, K, S, T).

```

1:  $k = 1, s = 1, t = 0$ ;
2:  $x' \leftarrow \text{VND}(x)$ ;
3: while  $k \leq K$  and  $t \leq T$  do
4:   while  $s \leq S$  do
5:      $x' \leftarrow \text{Shake}(x', s)$ ;
6:      $x' \leftarrow \text{VND}(x')$ ;
7:     if  $f(x') < f(x)$  then
8:        $x \leftarrow x'$ ;
9:        $s = 1$ ;
10:    else
11:       $s = s + 1$ ;
12:    end if
13:  end while
14:   $k = k + 1$ ;
15:   $s = 1$ ;
16:   $t \leftarrow \text{Run time}$ ;
17: end while
18: return  $x$ .

```

4.1. Generating an initial solution

A feasible solution for the SLOPP may be generated using any combination of a storage and a routing policy, such as those presented in Section 2.1.

The storage policy is used to assign products in $\mathcal{P}$ to locations in $\mathcal{L}$, i.e., to an aisle a and row b in the single-block layout. The policies implemented are: (i) random, (ii) dedicated with frequency-based and diagonal, and (iii) dedicated with frequency-based and within-aisle. Given a product $k \in \mathcal{P}$, its frequency is calculated by the number of times that $k \in Q_o$ for all $o \in \mathcal{O}$. In the diagonal policy, the distance between a slot located in $l = (a, b)$ and the I/O point is given by $d_{l0} = M(a - 1) + Nb$. Then, the product with the highest frequency is assigned to the closest slot to the I/O point, and so forth. Ties are broken randomly. For the within-aisle policy, the distance between a to the I/O point is calculated as $M(a - 1)$. The aisles are sorted from the closest to the farthest, and the closest aisle is filled with the products with the highest frequencies, and so on.

When the products locations are known, the SLOPP reduces to an OPP for each order. Each OPP can be solved using an optimal or a heuristic method. The objective function value of the solution is the sum of the distances traveled by the picker in all routes generated for the OPPs. In the storage location problem with heuristic routing policies, i.e., SL + Re, SL + Ss, SL + Mp and SL + Lg, the computation of the routes is straightforward, and done by verifying where picks are located in each aisle. We highlight that the largest gap requires additional operations to solve the maximum gap problem (Preparata and Shamos, 2012), which can be solved in $O(n)$ using the pigeonhole principle for n points of interest in the aisle, i.e., the items to be retrieved in it and the two end aisles. In the case of the SLOPP, the objective function is obtained by solving an instance of the travelling salesman problem (TSP) for each route, either using an optimal or heuristic method, such as LKH.

4.2. Local searches

We defined three neighborhoods to search for local improvements during VND. They are all based on the swap of products locations in the warehouse. Whenever the storage setup is changed, a new call for the routing policy is made to reevaluate the picking routes. A picking route is reevaluated only if a product contained in its pick list is involved in the change, and if the exchanged products do not belong to the same order, which would result in the same route.

The first neighborhood is defined as the swap of products assigned to different slots. The *slot exchange* LS finds two slots with an ordered product located in at least one of them and swaps their assignments. Mathematically, given a slot located in (a_k, b_k) containing a product k , i.e., $y_{a_k b_k}^k = 1$, and a slot located in (a_l, b_l) containing a product l , i.e., $y_{a_l b_l}^l = 1$, the slot exchange reassigns k to the slot in (a_l, b_l) , i.e., $y_{a_l b_l}^k = 1$, and l to the slot in (a_k, b_k) , i.e., $y_{a_k b_k}^l = 1$. All combinations of pairs of slots are tested except when $a_k = a_l$ and $b_k = b_l$. Since only slots containing products in different locations are swapped, this neighborhood size is $O(|\mathcal{P}||\mathcal{L}|)$ in the worst case, given that the number of locations $|\mathcal{L}| = AB$. A neighbor is an improving solution either if its objective function is better or, specially for this LS, when it has equal value, but the swap moves a product with higher frequency closer to the I/O point. This second condition proved to be very effective in avoiding bad local optimum.

The second neighborhood is defined as the swap of products contained in different rows. The *row exchange* LS finds two rows with at least one ordered product in at least one of them, even if they are in different aisles, and swaps their products. In the storage layout considered, the row exchange allows the exchange of up to four products simultaneously instead of two as in the slot exchange, which has the potential to escape from local optima found by the slot exchange. Mathematically, consider a row located in (a_i, b_i) containing products k and l , i.e., $y_{a_i b_i}^k = y_{a_i b_i}^l = 1$, and another row located in (a_j, b_j) containing products m and n , i.e., $y_{a_j b_j}^m = y_{a_j b_j}^n = 1$, the row exchange reassigns k and l to (a_j, b_j) , i.e., $y_{a_j b_j}^k = y_{a_j b_j}^l = 1$, and m and n to (a_i, b_i) , i.e., $y_{a_i b_i}^m = y_{a_i b_i}^n = 1$. The row exchange is applied for all pairs of rows in different locations with products assigned to them. This neighborhood size is $O(|\mathcal{L}|^2)$ in the worst case, i.e., when

Table 3

Warehouse layout and demand parameters used to generate the experimental instances.

Set	Aisles (A)	Rows per aisle (B)	Orders ($ O $)	Products per order ($ Q_o $)	Demand
Small	{1, 3, 5}	{5, 10}	{1, 5, 10}	{3, 5}	{Random}
Regular	{5, 10, 20}	{10, 50}	{10, 30, 50}	{5, 20, 50}	{Random, Skewed 50/30/20, Skewed 80/15/5}

there are products located in all existing rows.

The third neighborhood is defined as the swap of all products contained in different aisles. The *aisle exchange* LS finds two aisles with at least one ordered product in at least one of them and swaps their products. The position of the products in the original aisle is respected in their new aisle. This LS requires that the number of slots in the two swapped aisles are the same. Mathematically, for two swapped aisles $a_i, a_j \in \mathcal{A}$, given a product k located in (a_i, b) , i.e., $y_{aib}^k = 1$ and another product l located in (a_j, b) , i.e., $y_{ajb}^l = 1$, the aisle exchange reassigns k to (a_j, b) , i.e., $y_{ajb}^k = 1$, and l to (a_i, b) , i.e., $y_{aib}^l = 1$. This is done for all rows $b \in \mathbb{B}$ that belong to the two swapped aisles. For a warehouse with A identical aisles, this neighborhood size is $O(A^2)$ in the worst case.

5. Computational experiments

The SLAP and OPP literature lacks a well-established set of benchmark instances so that different methods could be compared on the same set. Instead, researchers usually create their own test instances based on the problem and the circumstances explored. We designed our instances using the parameters presented in Table 3 and share them on <https://www.leandro-coelho.com/slot-assignment-and-order-picking/>. All instances consider the distances as $M = N = 1$. The small set is used to evaluate the MIP models and to adjust the GVNS parameters, while the regular set, consisting of instances similar in size to those used in the literature (Pansart et al., 2018; Scholz and Wäscher, 2017; Scholz et al., 2016), is solved only by the GVNS. In Table 3, column *Aisles* represents the number of identical two-sided picking aisles in the storage area, *Rows per aisle* is the number of rows in each aisle, each one containing two slots, *Orders* corresponds to the number of orders to be fulfilled, and *Products per order* is the number of unique products contained in each order to be picked in one pick round. The orders were generated using three different distributions, as shown in the *Demand* column. The *Random* distribution selects the products using a uniform distribution, while the *Skewed* distribution uses the Pareto principle to divide products into three classes. The *Skewed A/B/C* notation is used to state that items from the smaller group appear A% of the times, the medium group appears B%, and the bigger group appears C%. We used A/B/C as either 50/30/20 or 80/15/5. Due to the low number of items in small instances, only the random distribution is used. Three different instances are created for each combination of these five parameters. The result is 108 small and 486 regular size instances. We highlight that the largest instances in the small set have the same parameters as the smallest instances in the regular set. However, they are generated with different seeds.

Computational experiments are performed in an Intel Gold 6148 Skylake CPU with 2.4 GHz and with a memory limit of 16 GB of RAM. The linear MIPs are solved using the CPLEX C++ API (version 12.5). The routes for the SLOPP are generated using the Concorde TSP solver. As reported by Theys et al. (2010), the LKH heuristic approximates very well the optimal OPP solution. Concorde provides an implementation of the LKH heuristic, which is used here. We use previously computed routes for an order as a warm start to LKH. Based on preliminary experiments, we fixed the number of 4-swap kicks in the LKH heuristic to two kicks. The heuristic routing policies are implemented in C++.

5.1. Solving the linear MIP model

The first set of experiments consists in solving instances in the small set using the linear MIP models for the SLOPP and its four special cases. The problems are solved using CPLEX with standard parameters and a time limit of two hours. The objective is to evaluate the maximum instance size for which an optimal solution can be proven using these models. For that, we group the instances from the small set according to the number of slots in the warehouse ($2AB$), and the number of items picked in a route ($|O||Q_o|$). Thus, each combination of Slots $\times$ Picks contains three instances. Table 4 shows the average time to solve the problem to optimality. Each cell is colored with a shade of grey representing the number of instances solved in that group, with darker cells representing more instances solved.

The results show that the linear MIP model for the general SLOPP is of no practical use even for small instances with only three or five picks, which have optimal solutions that could be found intuitively. We also highlight how fast the average time to prove optimality increases by adding more slots to the warehouse, even with only three picks. In larger instances, the SLOPP model runs out of memory, which is in line with Forrester (2016) about the standard linearization technique requiring a huge number of variables and constraints. The linearization of the cubic term in the objective function in the SLOPP model requires the introduction of $O(|O||\mathcal{L}|^2|\mathcal{P}|^2)$ new variables and constraints, which for large instances of the small set represents hundreds of millions of new variables.

When solving the models for the special cases, the solver was able to find optimality in most of the small instances. The SL + Re and SL + Ss models, which are relatively simpler in terms of number of variables and constraints, can be solved for instances slightly larger than the SL + Mp and SL + Lg models. We note that although there are cubic terms in some of the objective functions, they require the addition of only $O(|O||A|)$ new variables and constraints, significantly fewer than in the general SLOPP.

Table 4

Average time to prove optimality of small instances using the linear MIP models for the SLOPP and its four special cases.

Problem	Slots ($A \times B$)	Picks ($ \mathcal{O} \times \mathcal{Q}_o $)					
		3 (1×3)	5 (1×5)	15 (5×3)	25 (5×5)	30 (10×3)	50 (10×5)
SLOPP	10 (1×5)	0.3s	5.1s	-	-	-	-
	20 (1×10)	3.3s	-	-	-	-	-
	30 (3×5)	146.3s	-	-	-	-	-
	50 (5×5)	3435.6s	-	-	-	-	-
	60 (3×10)	7099.9s	-	-	-	-	-
	100 (5×10)	-	-	-	-	-	-
SL+Re	10 (1×5)	<0.1s	<0.1s	<0.1s	0.1s	3 0.1s	0.1s
	20 (1×10)	<0.1s	<0.1s	0.1s	0.9s	1.7s	22.8s
	30 (3×5)	<0.1s	<0.1s	0.8s	20.5s	83.1s	-
	50 (5×5)	<0.1s	<0.1s	4.6s	173.0s	181.9s	-
	60 (3×10)	<0.1s	<0.1s	0.7s	36.2s	216.5s	-
	100 (5×10)	<0.1s	0.1s	42.5s	294.3s	-	-
SL+Ss	10 (1×5)	<0.1s	<0.1s	<0.1s	0.1s	<0.1s	0.1s
	20 (1×10)	<0.1s	<0.1s	0.1s	0.7s	1.5s	21.7s
	30 (3×5)	<0.1s	<0.1s	1.0s	5.3s	296.3s	420.7s
	50 (5×5)	<0.1s	<0.1s	3.1s	21.6s	940.9s	-
	60 (3×10)	<0.1s	<0.1s	0.9s	15.1s	306.8s	-
	100 (5×10)	<0.1s	<0.1s	15.7s	57.0s	409.1s	-
SL+Mp	10 (1×5)	<0.1s	<0.1s	0.1s	0.2s	0.9s	1.4s
	20 (1×10)	<0.1s	<0.1s	1.2s	4.2s	10.7s	210.8s
	30 (3×5)	0.1s	0.1s	136.4s	1199.1s	-	-
	50 (5×5)	0.1s	0.2s	862.6s	4321.4s	-	-
	60 (3×10)	0.1s	0.2s	447.0s	3174.8s	-	-
	100 (5×10)	0.3s	0.9s	2984.6s	-	-	-
SL+Lg	10 (1×5)	<0.1s	<0.1s	0.4s	0.8s	2.5s	4.8s
	20 (1×10)	0.1s	0.1s	2.0s	25.4s	54.6s	6453.6s
	30 (3×5)	0.1s	0.2s	28.1s	71.6s	896.6s	-
	50 (5×5)	0.2s	0.3s	148.2s	387.2s	-	-
	60 (3×10)	0.4s	0.9s	155.1s	3018.8s	-	-
	100 (5×10)	1.5s	2.0s	656.0s	5044.5s	-	-

*Each cell contains the average over three instances. Dark gray shows that all three instances are solved. Medium gray indicates that only two instances are solved to optimality, and light gray indicates that only one instance is solved to optimality.

5.2. GVNS setting

Since the MIP models are difficult to solve for larger instances from the small set, an alternative method to deal with instances from the regular set is required. We now show how the GVNS was set for the experiments to solve the regular size set.

5.2.1. Setting the search strategy

In the neighborhood search strategy setting, we demonstrate through experiments the reason why the application of the three neighborhoods proposed in our VND is more effective than the application of only one or two of them. The experiment consists in creating initial solutions for the storage location subproblem of the SLOPP using the three storage policies implemented and by applying VND for each of them using different combinations of LSs. We tested an improvement strategy of using a single, a pair and a trio of LSs in all possible orders exhaustively, i.e., until no improving solution can be found. Results are compared for all instances of the small instance set for which the MIPs proved optimality. A table summarizing the results obtained is presented in A. The results indicate that the average gap significantly decreases when more neighborhoods are explored, when all three of them are performed, and the exploration order is not important (unlike when only two of them are used). For this reason, and to increase diversity, for the remaining experiments, we set the neighborhood exploration order to be randomly decided at the beginning of every VND. This leads to a reduction in the search bias during GVNS cycles, which helps avoid local minima. We also note that the exploration of the three neighborhoods is performed within one second, even for the largest instances of the set.

5.2.2. Setting the number of shakes and cycles

We solve the SLOPP and its special cases for all small instances using the proposed GVNS to improve solutions generated by the first VND. From the experiments described and with results reported in B, we set the maximum number of shakes $S = 5$. Then, each run is performed again up to $K = 20,000$ cycles. We calculate gaps to the BKS, either a proven optimal or the best feasible solution known, at different parts of the GVNS. The first solution observed is the initial solution. The next solution observed is that after VND

is applied to improve the initial solution (Algorithm 3, line 2). Then, we observe the current best solution for several values of K . From 10 different runs using different initial solutions generated using the storage policies implemented, a very fast convergence is observed when GVNS is at $K = 20$, so that the difference between solutions for all runs is no more than 0.01%. Since all solutions converge to similar ones regardless of the initial one, this is evidence of the robustness of this metaheuristic when improving solutions for the SLOPP and its special cases. The results of these tests indicate that solutions are improved up to $K = 10,000$, within a run time of less than two minutes. For these reasons, we choose this setting for the remaining experiments.

5.3. Solving the regular size set

We now perform detailed experiments to evaluate our algorithm on the regular size instance set. We analyze the solutions obtained for the SLOPP when different routing policies are used to create the picking routes. First, we use a method where solutions are evaluated during the whole search by a single routing policy. Four heuristic policies – return (Re), S-shape (Ss), midpoint (Mp), and largest gap (Lg) – and an optimal policy (LKH) are considered resulting in solutions for each of the five problems – SL + Re, SL + Ss, SL + Mp, SL + Lg and SLOPP, respectively – considered here. After, we present a new strategy that alternates the routing policy between Mp and LKH to show how the GVNS can benefit from the speed of the latter and the quality of the former to generate even better solutions for the SLOPP. The GVNS is run with the previous determined parameters ($S = 5$ and $K = 10,000$). The initial solution used is the best among the three generated from the storage policies. Each instance is solved five times. A two-hour time limit is used in all experiments since managers usually dispose of enough time to make rearrangement decisions when they are done, for example, on a daily basis. The results are presented for each problem setting, so that it is possible to analyze their impact in combination with each routing policy.

5.3.1. Comparison of solutions for the SLOPP and its special cases using GVNS with different routing policies

In this round of experiments, the objective is to compare the GVNS performance using different routing policies given the same run conditions, i.e., the same run time limit and parameters. We remind that a solution given by GVNS with any heuristic routing policy (Re, Ss, Mp, Lg) is both a solution for the special case and the integrated SLOPP, since the latter generalizes the former. From the results, we can draw conclusions about which special case gives better solutions under different instance conditions, and also for what routing policy the GVNS finds better solutions for the SLOPP. Table 5 shows the average route length given by GVNS with routes created using the five routing policies. The table is complemented with C, where we show the average run time and last cycle performed before the search stops. We have grouped all instances with different demand distributions in each row of the table, presenting the results for combinations of the remaining instance attributes. So, it is possible to identify for which routing policy the GVNS finds better solutions under each combination of warehouse size and orders' characteristics.

We conclude from the experiments that the best policy to generate routes for the SLOPP in our GVNS in most cases is Mp since these solutions consistently outperform the others. They have the lowest average route length and the largest number of best average solution for combinations of instance attributes. The second best is the Re, which presents a good performance, specially in warehouses with long aisles ($B = 50$). LKH finds some of the best solutions in smaller warehouses ($A = 5$ and $B = 10$), but significantly worse solutions in larger ones. Finally, Ss and Lg did not have the best average solution in any case.

The key factors to explain the success of midpoint compared to the other policies are: (i) low computational complexity; (ii) differentiated special cases; and (iii) good approximation of optimal routes.

Regarding factor (i), as shown by C, Mp average running time is slightly higher than those of Re and Ss, but lower than that of Lg and significantly lower than that of LKH. Lower computational complexity means that more solutions can be evaluated within the time limit, as shown by the higher value of k in C. Consequently, more neighborhoods are explored during the search, which increases the chances of finding the global optimum solution. The high computational complexity is the possible explanation of LKH performing well in many small instances ($A = 5$ and $B = 10$) and deteriorating fast when the instance becomes larger.

Factor (ii) differentiates the performance of Mp when compared to Lg. Normally, it is expected that Lg should dominate Mp (Dijkstra and Roodbergen, 2017), since it is a less restricted version of that. Since good SLOPP solutions tend to concentrate high demanded products close to the I/O point, it is common that solutions contain orders with all picks in two or more aisles of rows located across the front cross aisle. Examples of this are commonly found in instances with few picks ($Q_o = 5$) and long aisles ($B = 50$). In this case, Re generates a better route than Lg or Ss. However, when Q_o is higher, some orders may require that products are picked further into the aisles. In this case, it may be better to entirely cross both aisles, as one would under Ss and Lg. The more aisles a storage area has ($A = 20$), and the more routes are performed ($O = 50$), the greater are the chances that entirely crossing two aisles is better. Since Mp can provide good solutions to both situations, it has a higher chance of finding better solutions overall. Even if its special cases were incorporated to Lg, there is no guarantee that it would outperform Mp due to factor (i).

Factor (iii) differentiates Mp from Re and Ss. These two policies are known to perform well only under very special circumstances (Dijkstra and Roodbergen, 2017; Petersen, 1997; Van Gils et al., 2018b). The return policy approximates well an optimal route when picks are located either in only one aisle or in many aisles if they are in locations across the front cross aisle. Thus, Re is particularly good in warehouses with long aisles ($B = 50$) as shown by the experiments. Meanwhile, the S-shape policy performs best when there are many picks distributed along the entire storage area. Since optimal storage location assignments tend to concentrate high demanded products close to the I/O point, this situation arises when all stored products are highly demanded, such that every single order would contain picks in many different locations around the storage area. In our instances, this is the case of small warehouses ($A = 5$ and $B = 10$) with many products per order ($Q_o = 50$).

Other observations from the results are:

Table 5

Average route length found by GVNS with different routing policies.

A	B	O	Q_o	Re	Ss	Mp	Lg	LKH
5	10	10	5	108.9	144.7	108.9	144.7	109.2
			20	350.5	317.7	292.3	302.8	290.5
			50	717.3	542.8	549.0	558.4	522.0
		30	5	502.5	612.4	496.8	608.6	501.9
			20	1466.8	1250.5	1142.0	1174.9	1115.7
			50	2534.1	1872.4	1979.0	1940.9	1739.5
		50	5	999.5	1159.1	966.7	1146.0	968.5
			20	2694.8	2256.7	2080.0	2121.0	2018.8
			50	4412.8	3228.8	3468.0	3359.8	2994.6
	50	10	5	113.4	134.3	113.2	133.2	115.5
			20	374.2	808.4	372.9	822.6	708.1
			50	922.5	1143.1	947.9	1083.0	1089.1
		30	5	583.8	972.8	581.6	994.0	602.8
			20	2222.4	3102.0	2386.4	3134.1	2931.6
			50	4922.0	5224.7	4030.5	4347.0	4587.1
		50	5	1303.2	2264.4	1297.7	2264.8	1352.0
			20	4992.7	5986.1	4899.6	5613.6	5324.8
			50	10400.8	9629.5	7852.2	8379.5	8578.6
10	10	10	5	110.0	134.9	110.0	134.9	110.9
			20	365.6	358.5	320.4	323.9	323.8
			50	797.8	709.6	668.0	687.6	669.0
		30	5	514.9	606.2	511.7	604.2	522.3
			20	1685.7	1619.3	1441.3	1466.9	1414.4
			50	3265.1	2701.9	2632.9	2627.2	2526.6
		50	5	1058.6	1235.4	1048.8	1215.8	1063.8
			20	3298.3	3100.2	2718.4	2758.7	2674.4
			50	6044.4	4883.4	4833.7	4764.4	4562.5
	50	10	5	114.9	119.3	114.8	119.3	116.5
			20	325.2	826.3	325.0	824.8	925.4
			50	788.4	1167.6	807.7	1068.1	1566.4
		30	5	542.4	733.5	543.1	743.2	560.1
			20	1898.9	3261.7	1876.1	3251.6	3175.4
			50	4775.0	6662.5	4559.0	4666.0	5809.5
		50	5	1154.8	1726.9	1153.2	1757.6	1191.1
			20	4330.3	6549.2	4871.5	6029.2	5780.8
			50	10877.9	12987.1	8957.8	9730.3	11925.9
20	10	10	5	112.8	125.1	112.7	125.1	114.3
			20	360.8	378.7	336.5	337.3	349.7
			50	854.7	815.0	726.1	770.5	795.6
		30	5	525.2	585.0	521.8	582.0	535.8
			20	1813.4	1918.1	1623.8	1682.3	1657.9
			50	3845.7	3612.3	3308.8	3337.2	3317.5
		50	5	1100.0	1237.3	1086.0	1218.4	1118.7
			20	3697.3	3861.1	3226.8	3307.1	3331.5
			50	7572.4	6902.4	6323.3	6330.7	6256.9
	50	10	5	116.7	116.9	116.7	116.9	118.4
			20	321.7	723.5	321.9	757.3	1006.7
			50	788.2	1227.8	786.9	1077.9	1873.5
		30	5	557.2	580.3	558.1	588.0	565.3
			20	1638.4	3262.6	1629.2	3257.4	2807.9
			50	4209.3	7547.3	4526.6	4991.3	6864.4
		50	5	1163.6	1347.8	1163.2	1361.4	1184.5
			20	3710.3	6742.5	3696.5	6051.7	5775.2
			50	9874.4	15597.6	9500.8	10422.8	13777.4
		Average		2293.2	2715.1	2048.6	2355.9	2443.0

- instances with few picks and large warehouses (e.g., $A = 20$, $B = 50$, $O = 10$, $Q_o = 5$) have similar solutions regardless of the routing policy, since the optimal routes in those cases are the “go to the aisle, pick and return” special case. In these scenarios, picks from a single order are usually concentrated close to each other in a single aisle;
- in horizontal layouts, i.e., with a large number of short aisles ($A = 20$ and $B = 10$), and for a large number of picks ($Q_o = \{20, 50\}$), Lg and Mp perform best. In vertical layouts, i.e., with a small number of long aisles ($A = 5$ and $B = 50$), Mp and Re are the best. Possible explanations are such as those previously discussed for factor (ii);
- in warehouses with a similar layout proportion ($A = 5$ and $B = 10$ against $A = 20$ and $B = 50$) for the same number of picks, solutions are usually better in the smaller warehouse. The reason is that in our instances smaller warehouses have fewer varieties for ordered products and, consequently, a higher average demand per product, meaning a higher concentration of picks near the I/O point.

Table 6

Average solution improvement from the heuristics to an optimal storage policy for the SLOPP and its special cases.

		<i>SL + Re</i> Improv. over initial (%)	<i>SL + Ss</i> Improv. over initial (%)	<i>SL + Mp</i> Improv. over initial (%)	<i>SL + Ig</i> Improv. over initial (%)	<i>SLOPP</i> Improv. over initial (%)
A	5	40.7	28.8	42.1	33.7	35.1
	10	51.0	40.5	52.6	46.0	45.9
	20	59.0	50.2	58.7	53.8	53.8
B	10	38.1	32.9	41.0	35.8	33.1
	50	62.4	46.8	61.3	53.2	56.8
	10	56.5	40.9	55.9	45.3	50.5
O	30	49.6	41.0	50.7	45.9	44.2
	50	44.6	37.6	46.8	42.3	40.0
Q_0	5	53.6	55.0	53.3	53.4	52.1
	20	53.2	36.7	52.9	38.5	47.3
	50	44.0	27.9	47.1	41.5	35.4
D	Random	55.4	45.9	55.4	50.1	49.7
	Skew 50	51.9	41.1	52.8	46.6	46.4
	Skew 80	43.4	32.6	45.2	36.8	38.7

5.3.2. Solving the SLOPP using GVNS with a combination of Mp and LKH

From the previous experiments, we observe the robustness of Mp since it performs well in our GVNS under different instance settings. The results show that instead of wasting time evaluating routes for bad SLAP solutions, it is better to use a fast and fairly good heuristic to find better storage location assignments quicker, speeding up the improvement of the initial solution. However, it is expected that Mp can provide a good approximation to optimal OPP solutions, but not as good as LKH, given the same SLAP solution. Thus, we suggest here a different search strategy for our GVNS that alternates between Mp and LKH to solve the OPP sub-problem. This strategy consists of using Mp during the most intensive part of GVNS, i.e., the VND, and then update the solution using LKH to optimize the routes when VND is over. This way, the routes for the local optimum SLAP solution generated in the VND using Mp are updated to near optimal ones with LKH. We performed a new round of experiments for this new strategy. Detailed results are reported in D. In summary, it shows that the average solution provided by GVNS is improved from 2048.6 to 2023.5 (1.2% of improvement) when compared to the average solution of using only Mp. Since LKH requires more time to be solved, the average number of cycles to run this combined Mp/LKH is lower than when using only Mp. For that reason, there are situations where Mp alone may still outperform the combined Mp/LKH.

5.3.3. Solution improvement over the change of the storage policy

We analyze the potential improvement on route length by changing the storage policy from a heuristic to the optimal one considering that the routing policy is maintained. For each instance of the SLOPP, solved using the Mp/LKH strategy, and its four special cases, solved using the respective heuristic routing policy, we consider two solutions obtained in the experiments from Section 5.3.1 and 5.3.2: (i) the best initial solution among the three generated using a storage policy (random, diagonal and within-aisle); and (ii) the average solution of the five runs of GVNS for the storage location with the respective routing policy. In Table 6, we present the improvement from (i) to (ii) in column *Improv. over initial* calculated as $improv(x) = \frac{f(x) - f(x')}{f(x)}$, where x is the solution (i) and x' is the solution (ii). In each line of the table, average results are presented for the parameters shown on the left.

Two main conclusions are drawn from these results. First, regardless of the problem considered, the search for optimal storage location assignments significantly improves the solutions generated by the heuristic storage policies, with average improvements from 27.9% ($SL + Ss$ and $Q_0 = 50$) to 62.4% ($SL + Re$ and $B = 50$). Finally, the total distance traveled is improved more when optimal storage locations are searched, compared to using heuristic storage policies, when warehouses are larger, when there are fewer orders or orders with fewer items, and when demands are less skewed. This pattern is observed in almost all routing scenarios tested as bolded in the table.

6. Practical notes and further discussions

The results from research and practice show the importance of storage location and picker routing planning on the order picking performance. Traditionally, these problems are solved separately since the storage location has been considered to be more static in the literature and assumed to be less prone to changes. However, several factors, such as seasonality, marketing efforts, shelf life, and expiration dates, have an impact on the demand of the products. This means that the SLAP solution, in certain industries and for certain businesses, would be more dynamic and should be revised periodically. Some may argue that the cost of relocations is too high to justify product location reassignments being performed more often. However, considering the travelled distance minimization, the results of our experiments show the benefits of changing storage and routing policies in order to improve picking efficiency. Therefore, although known intuitively, warehouse managers should consider this link between the two problems, as modeled in our SLOPP.

Although it is interesting to see that mathematical models can be used to solve different versions of the SLOPP, the experiments show that they have limitations to solve even small instances. As our results show, the time spent to solve the problem increases dramatically as the size of either the warehouse or the orders grow. In reality, warehouse managers may deal with thousands of products in very large warehouses. The size of these real instances makes those exact methods impractical. Therefore, an application of a solution algorithm such as our GVNS not only saves time – as the unsolvable instances by the exact method are now solved within less than a second – but also provides good quality solutions in practice.

Obviously, when one changes from heuristic policies to optimal algorithms, it is expected to achieve better solutions. In this paper, we considered a case with single-block layout, two-sided picking and known orders where using the optimal storage layout we could on average reduce between about 27% to 62% of the total route length traveled by the pickers. Larger warehouses with fewer routes and smaller orders are especially fit for this change. From the warehouse managers point of view, the tradeoffs between reshuffling costs and the savings obtained by changing the current storage assignments to meet the SLOPP solution obtained by our GVNS need to be evaluated. Warehouse reshuffling policies and costs are discussed in Pazour and Carlo (2015). Since reshuffling can be performed in several ways depending upon the storage and retrieval system in use, this is an interesting topic for further investigation. Future research can consider the integration of SLOPP with the reshuffling problem for specific systems.

When rearrangements are performed less frequently than on a daily basis, say weekly or monthly, managers may dispose of much more time than the two-hour time limit considered in our experiments (Kofler et al., 2014). Since most of the runs were stopped before reaching the number of cycles so that solutions converge, this indicates that better solutions are still within reach if we could let our GVNS run for a longer time. In order to prove this claim, we ran experiments for the SLOPP using our GVNS with the Mp/LKH strategy for eight hours, and we observed an improvement of about 1% on average when compared to the two-hour limit (avg. solution reduced from 2023.5 to 2002.7), with improvements of up to 3% being found for larger instances. Nevertheless, the low improvement is an indication that convergence, and possibly the optimal solution for the problem, is not far. Finally, although not tested here, other combinations, possibly even more complex, can be developed for specific cases that may outperform both Mp and Mp/LKH for the SLOPP.

7. Conclusions

In this paper we have introduced new optimization tools to solve storage location assignment problems. The SLAP consists in arranging the products in a warehouse in order to optimize material handling during the picking process. A SLAP solution is evaluated according to the performance of the order picking, measured as the distance traveled by pickers. When decisions regarding product assignment and routing are integrated, we have the storage location and order picking problem (SLOPP) as introduced here. The SLOPP is modeled as a cubic mixed integer programming problem. We also model four special cases of the problem when routing policies are imposed, i.e., when the routes must be created using a heuristic routing policy, which are straightforward methods to create picking routes. These policies are modeled mathematically considering a typical single-block warehouse for the first time in this paper. To solve the SLOPP, we assume that all orders are known *a priori*, which might not be the case in markets with too volatile demands. Otherwise, when demands are known at the order level, the integration of these problems is advantageous.

Extensive computational experiments show that the models can deal with very limited size instances. As an alternative, we have proposed a General Variable Neighborhood Search (GVNS) metaheuristic with a descent phase consisting of the exploration of three different neighborhoods adapted for the problem. The GVNS presented is capable of providing optimal solutions for all instances solved using the models within less than a second. We also show that, in instances of size comparable to those of real scenarios, the search for optimal storage location assignments leads to savings ranging from about 27% to 62% compared to solutions from common storage policies. Savings can be even higher when considering that routing policies can also be changed, for example, from a heuristic policy to an optimal one. GVNS has also the potential to keep improving solutions the longer it runs. This demonstrates the potential benefits of searching for an optimal assignment of products to storage locations.

Research directions include strengthening or remodeling the formulations to speed up the search for the optimal solution in the single-block layout and other ones. Finally, other elements from warehouse planning may be incorporated to the SLOPP, such as the integration with other decisions (batching, zoning), constraints (safety, blocking) and parameters (rearrangement costs to modify the storage plan).

CRedit authorship contribution statement

Allyson Silva: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Visualization. **Leandro C. Coelho:** Conceptualization, Methodology, Validation, Resources, Writing - original draft, Writing - review & editing, Supervision, Project administration, Funding acquisition. **Maryam Darvish:** Conceptualization, Methodology, Validation, Resources, Writing - original draft, Writing - review & editing, Supervision, Project administration, Funding acquisition. **Jacques Renaud:** Conceptualization, Methodology, Validation, Writing - original draft, Writing - review & editing.

Appendix A. Neighborhood exploration order

Table A.7 presents the results for all rounds of experiments with different neighborhood exploration orders. Column *LS order* shows the exploration order of the neighborhoods. The subsequent columns show the average gap of the VND solution to the

Table A.7

Results for the exploration of different combinations of neighborhoods in a VND.

<i>LS order</i>	<i>SLOPP</i> <i>Gap (%)</i>	<i>SL + Re</i> <i>Gap (%)</i>	<i>SL + Ss</i> <i>Gap (%)</i>	<i>SL + Mp</i> <i>Gap (%)</i>	<i>SL + Lg</i> <i>Gap (%)</i>	<i>Avg.</i>
<i>Slot</i>	0.00	5.84	8.50	3.53	6.67	6.02
<i>Row</i>	9.06	19.44	21.67	16.22	19.55	18.95
<i>Aisle</i>	21.97	33.38	36.01	27.70	33.31	32.40
<i>Slot → Row</i>	0.00	4.16	7.50	2.41	5.81	4.89
<i>Slot → Aisle</i>	0.00	5.61	6.47	3.39	4.68	4.93
<i>Row → Slot</i>	0.00	4.51	7.45	2.91	5.76	5.06
<i>Row → Aisle</i>	8.51	18.71	20.57	15.49	18.44	18.04
<i>Aisle → Slot</i>	0.00	5.89	6.91	3.47	4.49	5.10
<i>Aisle → Row</i>	8.51	18.54	20.67	15.15	18.04	17.86
<i>Slot → Row → Aisle</i>	0.00	3.91	5.49	2.34	3.80	3.82
<i>Slot → Aisle → Row</i>	0.00	3.90	5.51	2.34	3.84	3.83
<i>Row → Slot → Aisle</i>	0.00	4.28	5.37	2.80	3.91	4.00
<i>Row → Aisle → Slot</i>	0.00	4.26	5.53	2.78	3.95	4.04
<i>Aisle → Slot → Row</i>	0.00	3.91	5.50	2.17	3.66	3.76
<i>Aisle → Row → Slot</i>	0.00	4.09	5.46	2.82	3.76	3.95
<i>Initial solution avg. gap (%)</i>	22.33	33.61	36.45	28.19	33.84	32.81

optimum found by the MIP for each of the five problems for the instances solved up to optimality with the best gaps in bold, and column Avg. is the weighted mean of the gaps with the number of instances in each problem as weights. To compare, we show the average gap of the three initial solutions generated by each of the storage policies to the optimal solution found by the MIP in row *Initial solution avg. gap (%)*. Computation times are not reported since all VND calls took less than a second to finish.

Appendix B. Setting the maximum number of shakes

We test S for different values to find if there is an appropriate one that leads to better solutions. The GVNS was tested with $S = \{1, 2, 4, 5, 10, 20, 40\}$. Since VND is the most costly step in our algorithm, we decided to use $K = 400/S$ in order to keep the number of VND calls approximately constant in each run. Each instance of the small set is solved 10 times, starting from each of the three storage policies solution for each one of the five problems considered. Table B.8 reports the results found. They are divided into two sets. *Optimal* is for the instances solved up to optimality by CPLEX, while *Feasible* is for those with an upper bound solution known but with optimality unproven within the time limit of two hours. Results for the integrated SLOPP solved using the GVNS scheme are not reported since the optimal was already found by only using a single VND, as shown previously in Table A.7, and there is not a significant number of instances in the *Feasible* category for this problem. The results show that the GVNS performance for $S = 5$ is, on average, better than for other values.

Table B.8Results for the parameter tuning of the maximum number of shakes S .

	S	K	<i>SL + Re</i>		<i>SL + Ss</i>		<i>SL + Mp</i>		<i>SL + Lg</i>	
			<i>Gap (%)</i>	<i>Time (s)</i>	<i>Gap (%)</i>	<i>Time (s)</i>	<i>Gap (%)</i>	<i>Time (s)</i>	<i>Gap (%)</i>	<i>Time (s)</i>
<i>Optimal</i>	1	400	0.03	0.4	0.02	0.4	0.00	0.3	0.01	0.5
	2	200	0.02	0.4	0.00	0.5	0.00	0.3	0.00	0.5
	4	100	0.01	0.4	0.00	0.5	0.00	0.4	0.00	0.6
	5	80	0.00	0.5	0.00	0.5	0.00	0.4	0.00	0.6
	10	40	0.00	0.5	0.00	0.6	0.00	0.4	0.00	0.7
	20	20	0.00	0.5	0.00	0.6	0.00	0.4	0.00	0.7
	40	10	0.00	0.6	0.00	0.7	0.00	0.5	0.00	0.8
	#Inst		92		99		75		80	
<i>Feasible</i>	1	400	-0.28	1.5	0.99	1.9	-3.34	1.6	-5.70	2.5
	2	200	-0.40	1.7	0.58	2.1	-3.50	1.8	-5.97	2.7
	4	100	-0.52	1.8	0.43	2.4	-3.59	1.9	-6.12	3.1
	5	80	-0.58	1.9	0.35	2.5	-3.59	2.0	-6.11	3.2
	10	40	-0.42	2.2	0.49	2.9	-3.58	2.3	-6.09	3.8
	20	20	-0.36	2.5	0.63	3.5	-3.53	2.6	-5.95	4.5
	40	10	-0.32	2.7	0.95	4.1	-3.49	2.9	-5.77	5.2
	#Inst		16		9		33		28	

Appendix C. Average time and number of cycles when solving the regular set

Table C.9 complements Table 5 by presenting the average running time of GVNS when stopped by one of the stopping criteria (7200 s or $K = 10,000$ cycles) and the last cycle k performed during the search. Time may be above 7,200 s due to the placement of the verification of the time stopping condition in the algorithm. Rows that neither show a time of 7200 s nor $k = 10,000$ indicate that instances within the group were stopped by different criteria between both.

Table C.9

Average running time and last cycle performed by GVNS for the SLOPP with different routing policies.

A	B	O	Q _o	<u>Re</u>		<u>Ss</u>		<u>Mp</u>		<u>Lg</u>		<u>LKH</u>				
				Time (s)	k	Time (s)	k	Time (s)	k	Time (s)	k	Time (s)	k			
5	10	10	5	374	10000	504	10000	374	10000	406	10000	7201	736			
			20	1379	7858	2251	10000	1379	10000	1547	10000	7226	24			
			50	3411	2534	5546	10000	3411	10000	3059	10000	7335	3			
			30	5	1110	10000	1598	10000	1110	10000	1358	10000	7203	215		
				20	4626	2337	7074	9894	4626	10000	5121	10000	7278	5		
				50	7200	750	7200	3361	7200	6133	7200	7367	7705	1		
			50	5	1868	9976	2712	10000	1868	10000	2236	10000	7204	119		
				20	7200	1314	7200	5521	7200	8790	7200	8353	7353	2		
				50	7200	413	7200	1682	7200	3450	7200	4181	8239	1		
		50	10	5	5220	10000	5858	10000	5220	10000	4537	10000	7204	135		
				20	7200	1699	7200	4743	7200	6086	7200	6474	7899	1		
				50	7200	404	7201	1549	7200	1891	7200	2436	7577	1		
			30	5	7200	3766	7200	4566	7200	5432	7200	6030	7220	31		
				20	7201	438	7201	1278	7201	1626	7200	1865	7692	1		
				50	7201	78	7201	367	7201	457	7202	613	7200	1		
			50	5	7200	1861	7200	2616	7200	3143	7200	3151	7227	15		
				20	7201	196	7201	654	7201	885	7201	949	7262	1		
				50	7203	32	7204	160	7203	252	7202	312	7200	1		
		10	10	10	5	1095	10000	1401	10000	1095	10000	1106	10000	7202	324	
					20	3316	2533	5829	10000	3316	10000	4157	10000	7267	7	
					50	7200	595	7200	4967	7200	6754	7200	6505	7750	1	
					30	5	2883	8218	3964	10000	2883	10000	3440	10000	7206	95
						20	7200	674	7200	3808	7200	6508	7200	5403	7431	1
						50	7200	162	7201	1206	7200	2040	7200	1910	7335	1
					50	5	4834	4182	6905	9899	4834	10000	5913	10000	7210	52
						20	7200	390	7200	2152	7200	3670	7200	3118	7618	1
						50	7201	88	7201	588	7201	1103	7201	1131	7201	1
				50	10	5	7200	3692	7200	3517	7200	3767	7200	4529	7213	62
						20	7200	681	7200	1909	7200	2333	7200	2220	8106	1
						50	7201	154	7201	699	7201	931	7201	875	7683	1
					30	5	7200	1504	7200	1534	7200	1717	7200	2049	7235	17
						20	7201	196	7201	562	7201	741	7201	684	7430	1
						50	7203	14	7204	158	7203	219	7203	243	7202	1
					50	5	7201	805	7201	896	7201	1017	7201	1155	7259	8
						20	7201	82	7203	301	7201	385	7202	382	7200	1
						50	7205	2	7208	48	7205	95	7204	127	7201	1
20	10			10	5	3692	10000	4442	10000	3692	10000	3376	10000	7205	156	
					20	7200	1184	7200	4972	7200	7989	7200	6906	7435	2	
					50	7200	182	7200	1742	7200	2611	7200	2335	7222	1	
					30	5	7200	3791	7200	6511	7200	8381	7200	7662	7212	45
						20	7200	238	7200	1468	7200	2746	7200	2210	7966	1
						50	7201	38	7201	413	7201	828	7201	669	7209	1
					50	5	7200	1882	7200	3704	7200	5026	7200	4318	7232	20
						20	7200	117	7201	803	7200	1550	7201	1220	8117	1
						50	7201	17	7203	202	7201	432	7202	385	7206	1
				50	10	5	7201	1074	7201	925	7201	962	7200	1211	7225	30
						20	7201	273	7201	540	7201	635	7201	675	8553	1
						50	7202	74	7203	247	7202	324	7202	312	7201	1
					30	5	7201	497	7201	438	7201	474	7201	599	7272	8
						20	7202	88	7203	222	7202	282	7202	218	7208	1
						50	7206	5	7211	67	7206	94	7207	80	7205	1
					50	5	7202	276	7203	267	7202	289	7202	342	7347	4
						20	7204	33	7205	112	7204	149	7205	115	7211	1
						50	7212	1	7217	18	7212	30	7210	36	7214	1
		Average			6227	4115	6272	4099	6492	3543	6873	2174	7393	40		

Appendix D. Results for the SLOPP using GVNS with the Mp/LKH search strategy

Table D.10 presents the results for the experiments with GVNS using the combined Mp/LKH routing creation strategy as described in Section 5.3.2 for the instances of the regular set.

Table D.10

Results for the SLOPP using GVNS with the Mp/LKH search strategy

B	O	Q ₀	A = 5			A = 10			A = 20		
			Avg. sol.	Time (s)	k	Avg. sol.	Time (s)	k	Avg. sol.	Time (s)	k
10	10	5	108.9	539	10000	110	1485	10000	112.7	4637	10000
		20	292.3	2350	10000	319.6	5971	10000	336	7200	4921
		50	521.3	5585	10000	662.5	7200	5073	724.6	7201	1816
	30	5	496.8	1719	10000	512	4234	10000	521.6	7200	6090
		20	1136.7	7146	9888	1437.3	7200	3878	1624.4	7200	1507
		50	1741.5	7200	4537	2512.7	7200	1662	3277.6	7201	531
	50	5	966	2893	10000	1049.3	7090	9589	1085.2	7200	3528
		20	2050.1	7200	5800	2707.4	7200	2268	3225.7	7201	842
		50	2997.4	7200	2578	4523.8	7201	865	6207.6	7203	279
	50	5	113.2	6164	10000	114.9	7200	3347	116.8	7201	880
		20	374.3	7200	4314	324.8	7200	1800	321.7	7201	524
		50	951.7	7201	1488	809.5	7201	649	790.4	7202	232
	30	5	582.7	7200	4286	543.2	7201	1400	557.6	7202	398
		20	2383.6	7201	1227	1877	7201	523	1638.6	7203	208
		50	4039.2	7202	406	4557.1	7204	164	4536.8	7209	69
	50	5	1297.8	7200	2417	1152.7	7201	825	1161.8	7202	246
		20	4902	7201	635	4897.9	7203	291	3698.4	7205	104
		50	7795.5	7204	194	8931.1	7208	57	9538.6	7222	21

*Avg. sol. = 2023.5; Avg. time = 6524s; Avg. k = 3562.

References

- Applegate, D.L., Bixby, R.E., Chvatal, V., Cook, W.J., 2007. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, Princeton.
- Boysen, N., Stephan, K., 2013. The deterministic product location problem under a pick-by-order policy. *Discrete Appl. Math.* 161 (18), 2862–2875.
- Carlo, H.J., Giraldo, G.E., 2012. Toward perpetually organized unit-load warehouses. *Comput. Industr. Eng.* 63 (4), 1003–1012.
- Chabot, T., Coelho, L.C., Renaud, J., Côté, J.-F., 2018. Mathematical model, heuristics and exact method for order picking in narrow aisles. *J. Oper. Res. Soc.* 69 (8), 1242–1253.
- Chan, F.T.S., Chan, H.K., 2011. Improving the productivity of order picking of a manual-pick and multi-level rack distribution warehouse through the implementation of class-based storage. *Expert Syst. Appl.* 38 (3), 2686–2700.
- Chen, C.-M., Gong, Y., De Koster, R.B.M., Van Nunen, J.A.E.E., 2010. A flexible evaluative framework for order picking systems. *Prod. Oper. Manage.* 19 (1), 70–82.
- Chen, L., Langevin, A., Riopel, D., 2011. A tabu search algorithm for the relocation problem in a warehousing system. *Int. J. Prod. Econ.* 129 (1), 147–156.
- Chuang, Y.-F., Lee, H.-T., Lai, Y.-C., 2012. Item-associated cluster assignment model on storage allocation problems. *Comput. Industr. Eng.* 63 (4), 1171–1177.
- Dekker, R., De Koster, R.B.M., Roodbergen, K.J., Van Kalleveen, H., 2004. Improving order-picking response time at Ankor's warehouse. *Interfaces* 34 (4), 303–313.
- De Koster, R.B.M., Le-Duc, T., Roodbergen, K.J., 2007. Design and control of warehouse order picking: A literature review. *Eur. J. Oper. Res.* 182 (2), 481–501.
- Dijkstra, A.S., Roodbergen, K.J., 2017. Exact route-length formulas and a storage location assignment heuristic for picker-to-parts warehouses. *Transp. Res. Part E: Logist. Transp. Rev.* 102, 38–59.
- Drezner, Z., 2008. Extensive experiments with hybrid genetic algorithms for the solution of the quadratic assignment problem. *Comput. Oper. Res.* 35 (3), 717–736.
- Forrester, R.J., 2016. Tightening concise linear reformulations of 0–1 cubic programs. *Optimization* 65 (4), 877–903.
- Frazelle, E., 2016. *World-Class Warehousing and Material Handling*, second ed. McGraw-Hill, New York.
- Glover, F., Woolsey, E., 1974. Converting the 0–1 polynomial programming problem to a 0–1 linear program. *Oper. Res.* 22 (1), 180–182.
- Gu, J., Goetschalckx, M., McGinnis, L.F., 2007. Research on warehouse operation: A comprehensive review. *Eur. J. Oper. Res.* 177 (1), 1–21.
- Gu, J., Goetschalckx, M., McGinnis, L.F., 2010. Research on warehouse design and performance evaluation: A comprehensive review. *Eur. J. Oper. Res.* 203 (3), 539–549.
- Hansen, P., Mladenović, N., Brimberg, J., Pérez, J.A.M., 2019. Variable neighborhood search. In: Gendreau, M., Potvin, J.-Y. (Eds.), *Handbook of Metaheuristics*. Springer, New York, pp. 57–97.
- Hausman, W.H., Schwarz, L.B., Graves, S.C., 1976. Optimal storage assignment in automatic warehousing systems. *Manage. Sci.* 22 (6), 629–638.
- Helsgaun, K., 2000. An effective implementation of the Lin-Kernighan traveling salesman heuristic. *Eur. J. Oper. Res.* 126 (1), 106–130.
- Hsieh, L.-F., Huang, Y.-C., 2011. New batch construction heuristics to optimise the performance of order picking systems. *Int. J. Prod. Econ.* 131 (2), 618–630.
- Hsieh, L.-F., Tsai, L., 2006. The optimum design of a warehouse system on order picking efficiency. *Int. J. Adv. Manuf. Technol.* 28 (5–6), 626–637.
- Kim, B.S., Smith, J.S., 2012. Slotting methodology using correlated improvement for a zone-based carton picking distribution system. *Comput. Industr. Eng.* 62 (1), 286–295.
- Kofler, M., Beham, A., Wagner, S., Affenzeller, M., 2014. Affinity based slotting in warehouses with dynamic order patterns. In: *Advanced Methods and Applications in Computational Intelligence*. Springer, pp. 123–143.
- Li, J., Moghaddam, M., Nof, S.Y., 2016. Dynamic storage assignment with product affinity and ABC classification – a case study. *Int. J. Adv. Manuf. Technol.* 84 (9–12), 2179–2194.
- Mantel, R.J., Schuur, P.C., Heragu, S.S., 2007. Order oriented slotting: A new assignment strategy for warehouses. *Eur. J. Industr. Eng.* 1 (3), 301–316.
- Ming-Huang Chiang, D., Lin, C.-P., Chen, M.-C., 2014. Data mining based storage assignment heuristics for travel distance reduction. *Expert Syst.* 31 (1), 81–90.
- Mladenović, N., Hansen, P., 1997. Variable neighborhood search. *Comput. Oper. Res.* 24 (11), 1097–1100.

- Nagy, G., Salhi, S., 2007. Location-routing: Issues, models and methods. *Eur. J. Oper. Res.* 177 (2), 649–672.
- Pan, J.C.-H., Wu, M.-H., 2012. Throughput analysis for order picking system with multiple pickers and aisle congestion considerations. *Comput. Oper. Res.* 39 (7), 1661–1672.
- Pansart, L., Catusse, N., Cambazard, H., 2018. Exact algorithms for the order picking problem. *Comput. Oper. Res.* 100, 117–127.
- Pazour, J.A., Carlo, H.J., 2015. Warehouse reshuffling: Insights and optimization. *Transp. Res. Part E: Logist. Transp. Rev.* 73, 207–226.
- Petersen, C.G., 1997. An evaluation of order picking routing policies. *Int. J. Oper. Prod. Manage.* 17 (11), 1098–1111.
- Petersen, C.G., 1999. The impact of routing and storage policies on warehouse efficiency. *Int. J. Oper. Prod. Manage.* 19 (10), 1053–1064.
- Petersen, C.G., Aase, G.R., 2004. A comparison of picking, storage, and routing policies in manual order picking. *Int. J. Prod. Econ.* 92 (1), 11–19.
- Petersen, C.G., Aase, G.R., Heiser, D.R., 2004. Improving order-picking performance through the implementation of class-based storage. *Int. J. Phys. Distrib. Logist. Manage.* 34 (7), 534–544.
- Preparata, F.P., Shamos, M.I., 2012. *Computational Geometry: An Introduction*. Springer Science & Business Media, New York.
- Quader, S., Castillo-Villar, K.K., 2018. Design of an enhanced multi-aisle order-picking system considering storage assignments and routing heuristics. *Robot. Comput. Integr. Manuf.* 50, 13–29.
- Ratliff, H.D., Rosenthal, A.S., 1983. Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem. *Oper. Res.* 31 (3), 507–521.
- Reyes, J., Solano-Charris, E., Montoya-Torres, J., 2019. The storage location assignment problem: A literature review. *Int. J. Industr. Eng. Comput.* 10 (2), 199–224.
- Roodbergen, K.J., De Koster, R.B.M., 2001. Routing methods for warehouses with multiple cross aisles. *Int. J. Prod. Res.* 39 (9), 1865–1883.
- Roodbergen, K.J., Vis, I.F.A., Taylor Jr., G.D., 2015. Simultaneous determination of warehouse layout and control policies. *Int. J. Prod. Res.* 53 (11), 3306–3326.
- Salhi, S., Rand, G.K., 1989. The effect of ignoring routes when locating depots. *Eur. J. Oper. Res.* 39 (2), 150–156.
- Scholz, A., Wäscher, G., 2017. Order batching and picker routing in manual order picking systems: The benefits of integrated routing. *Cent. Eur. J. Oper. Res.* 25 (2), 491–520.
- Scholz, A., Henn, S., Stuhlmann, M., Wäscher, G., 2016. A new mathematical programming formulation for the single-picker routing problem. *Eur. J. Oper. Res.* 253 (1), 68–84.
- Tappia, E., Roy, D., Melacini, M., De Koster, R.B.M., 2019. Integrated storage-order picking systems: Technology, performance models, and design insights. *Eur. J. Oper. Res.* 274 (3), 947–965.
- Theys, C., Bräysy, O., Dullaert, W., Raa, B., 2010. Using a TSP heuristic for routing order pickers in warehouses. *Eur. J. Oper. Res.* 200 (3), 755–763.
- Van Gils, T., Caris, A., Ramaekers, K., 2018a. Reducing picker blocking in a high-level narrow-aisle order picking system. In: 2018 Winter Simulation Conference (WSC). IEEE, pp. 2953–2965.
- Van Gils, T., Ramaekers, K., Braekers, K., Depaire, B., Caris, A., 2018b. Increasing order picking efficiency by integrating storage, batching, zone picking, and routing policy decisions. *Int. J. Prod. Econ.* 197, 243–261.
- Van Gils, T., Ramaekers, K., Caris, A., De Koster, R.B.M., 2018c. Designing efficient order picking systems by combining planning problems: State-of-the-art classification and review. *Eur. J. Oper. Res.* 267 (1), 1–15.
- Van Gils, T., Caris, A., Ramaekers, K., Braekers, K., de Koster, R.B.M., 2019. Designing efficient order picking systems: The effect of real-life features on the relationship among planning problems. *Transp. Res. Part E: Logist. Transp. Rev.* 125, 47–73.
- Weidinger, F., 2018. Picker routing in rectangular mixed shelves warehouses. *Comput. Oper. Res.* 95, 139–150.
- Yener, F., Yazgan, H.R., 2019. Optimal warehouse design: Literature review and case study application. *Comput. Industr. Eng.* 129, 1–13.